

Quality-Stratified Low-Rank Adaptation for Fine-Tuning Large Language Models on Private Document Collections

A Thesis

Presented in Partial Fulfillment of the Requirements for the Degree
Master of Science in the Graduate School of The Ohio State University

By

Pratham Sharma, M.S

Graduate Program in Department of Computer Science and Engineering

The Ohio State University

2026

Master's Examination Committee:

Dr. Hari Subramoni, Advisor

Dr. Rajiv Ramnath

Dr. Scott Shearer

© Copyright by

Pratham Sharma

2026

Abstract

Fine-tuning large language models on private document collections requires supervision from synthetically generated question-answer pairs, since human-annotated data is unavailable for proprietary corpora. Synthetic pipelines produce training pairs of highly variable quality, yet existing parameter-efficient fine-tuning methods treat all pairs equally.

This thesis proposes Quality-Stratified Low-Rank Adaptation (QS-LoRA), a fine-tuning pipeline that scores each synthetic pair on four quality dimensions, assigns pairs to tiers, trains a separate LoRA adapter per tier at rank proportional to quality ($r = 32, 16, 8$), and merges the three adapters offline into a single dense checkpoint. The method is designed as a quality-aware PEFT pipeline for constrained on-premise deployment, not a new general PEFT paradigm. The novelty is the specific integration of quality scoring, tiered rank allocation, and offline merging, validated empirically across diverse model families and document-grounded benchmarks.

Evaluated across 29 model-dataset combinations spanning ten backbone models and three benchmarks, QS-LoRA outperforms uniform LoRA in every cell (mean +0.14 GA, median +0.14 GA, scale 1–5). At 1B–8B, QS-LoRA outperforms both uniform LoRA and AdaLoRA in all 21 evaluated cells. At 12B and above, AdaLoRA outperforms QS-LoRA; this performance reversal is the main practical limitation of the method’s applicability.

All results reflect single-seed training runs. Item-level bootstrap confidence intervals are provided but do not capture training variance across seeds. Claims should be interpreted as evidence from a carefully controlled single-seed study.

To my family, for their unwavering support throughout this journey.

Acknowledgments

This work would not have been possible without the support and guidance of many people throughout my time at Ohio State.

I am deeply grateful to Dr. Hari Subramoni for taking me on as a graduate student and for the direction he provided at every stage of this research. His ability to ask the right questions at the right time consistently helped me see my own work more clearly. Being part of SAILAB has been one of the most formative experiences of my graduate career, and I am thankful for everything that came with it.

I would like to thank Dr. Rajiv Ramnath for serving on my thesis committee and for the time he devoted to reviewing this work. His perspective during the examination process was genuinely valuable.

I am also grateful to Dr. Scott Shearer for broadening how I think about research. Working with him exposed me to problems and approaches I would not have encountered otherwise, and that breadth has made me a better researcher.

To my labmates Anirudh Potlapally and Aswathnarayan Radhakrishnan – thank you. The conversations we had about research, industry, and everything in between shaped this work in ways that are hard to fully articulate. I am lucky to have had colleagues who were also willing to be sounding boards, critics, and friends.

Vita

2024 B.Tech. in Computer Science and Engineering, Vellore Institute of Technology, India

2024–Present Graduate Research Associate, SAI Lab, The Ohio State University

Fields of Study

Major Field: Computer Science and Engineering

Studies in: Natural Language Processing, Parameter-Efficient Fine-Tuning, Large Language Models, and Machine Learning Systems

Table of Contents

	Page
Abstract	ii
Dedication	iv
Acknowledgments	v
Vita	vi
List of Tables	xi
List of Figures	xvi
1. Introduction	1
1.1 Motivation	1
1.1.1 Scope of Claims	4
1.2 Problem Statement	5
1.3 Research Objectives	6
1.4 Contributions	7
1.5 Thesis Organization	9
2. Background and Related Work	11
2.1 Fine-Tuning of Large Language Models	11
2.1.1 Full Fine-Tuning	12
2.1.2 Instruction Tuning	13
2.1.3 Domain-Specific Fine-Tuning	13
2.2 Parameter-Efficient Fine-Tuning	14
2.2.1 Adapter Layers	14
2.2.2 Prefix Tuning and Prompt Tuning	15
2.2.3 Sparse and Selective Fine-Tuning	16

2.3	Low-Rank Adaptation and Its Variants	16
2.3.1	LoRA: Low-Rank Adaptation	17
2.3.2	AdaLoRA: Adaptive Budget Allocation	17
2.3.3	QLoRA: Quantized Low-Rank Adaptation	18
2.3.4	DoRA: Weight-Decomposed Low-Rank Adaptation	19
2.3.5	Other LoRA Extensions	19
2.4	Retrieval-Augmented Generation	20
2.4.1	Architecture and Core Components	21
2.4.2	Limitations of Retrieval-Augmented Generation	21
2.4.3	Fine-Tuning vs. Retrieval-Augmented Generation	22
2.5	Synthetic Question-Answer Generation	23
2.5.1	Self-Supervised and Model-Generated Instruction Data	23
2.5.2	Document-Grounded Question-Answer Generation	24
2.5.3	Quality Filtering of Synthetic Data	25
2.6	LLM-as-Judge for Automated Evaluation and Quality Scoring	25
2.6.1	Foundations of LLM-Based Evaluation	26
2.6.2	Known Biases in LLM Evaluation	26
2.6.3	Rubric-Based Pointwise Scoring	27
2.7	Merging of Fine-Tuned Adapters	28
2.7.1	Parameter-Space Merging of Full Models	28
2.7.2	Interference in Multi-Task Merging	29
2.7.3	LoRA Adapter Merging	30
3.	Quality-Stratified Low-Rank Adaptation	31
3.1	Pipeline Overview	31
3.2	Synthetic Question-Answer Generation	32
3.2.1	Generator Model	33
3.2.2	Generation Prompt	35
3.2.3	Document Preparation	35
3.3	Quality Scoring	36
3.3.1	Scoring Rubric	37
3.3.2	Score Distribution	38
3.4	Tier Assignment	38
3.5	Tier-Specific LoRA Adapter Training	41
3.5.1	Rank and Scaling	41
3.5.2	Target Modules	42
3.5.3	Training Configuration	43
3.6	Tier-Weighted Dense Merge	44
3.6.1	Merge Formula	44
3.6.2	Inference Efficiency	46
3.7	Distilled Local Judge	46

3.7.1	Training	47
3.7.2	Evaluation	47
4.	Experimental Setup	49
4.1	Evaluation Datasets	49
4.1.1	RepLiQA	49
4.1.2	Quoref	51
4.1.3	SQuAD v2	51
4.2	Baseline Conditions	52
4.2.1	B3: Uniform LoRA	53
4.2.2	B5: AdaLoRA	53
4.2.3	Ours: QS-LoRA	54
4.3	Evaluation Metric	54
4.3.1	Gold Alignment	54
4.3.2	Why Gold Alignment Over Lexical Metrics	55
4.3.3	Bootstrap Confidence Intervals	56
4.4	Backbone Models	57
4.5	Hardware and Infrastructure	58
5.	Results and Analysis	60
5.1	Results: Small Models (1B–8B)	60
5.1.1	RepLiQA	61
5.1.2	Quoref	62
5.1.3	SQuAD v2	63
5.2	Results: Large Models (12B–70B)	64
5.2.1	RepLiQA	64
5.2.2	Quoref	65
5.2.3	SQuAD v2	66
5.3	Ceiling Gap Analysis	67
5.4	Training Cost Analysis	69
5.5	Adapter Weight Geometry Analysis	71
5.6	Ablation Studies	75
5.6.1	Merge Weight Scheme	76
5.6.2	Cross-Model Merge Weight Generalization	78
5.6.3	Low-Quality Pair Contribution	80
5.6.4	Generator Quality: Fine-Tuned vs API Generator	82
5.6.5	Isolating Stratification from Rank Allocation	83
5.7	Distilled Judge Evaluation	85
5.8	Hallucination Analysis	86
5.8.1	Metric: Unsupported-Answer Rate	87

5.8.2	Quantitative Results	87
5.8.3	Coverage and Abstention Behavior	89
5.8.4	Qualitative Examples	91
5.9	Effect of Fine-Tuning on General Capabilities	94
6.	Conclusion	96
6.1	Summary of Contributions	96
6.2	Key Findings	99
6.3	Limitations	102
6.4	Future Work	104
	Bibliography	108

List of Tables

Table		Page
1.1	Actual training cost for one fine-tuning run (QS-LoRA, RepLiQA dataset) on AWS-equivalent A100-SXM4-80GB hardware at \$5.12/hr per GPU. Times measured empirically in this thesis. 70B requires 4× A100 GPUs; all other models use 1× GPU.	3
3.1	Quality tier distribution after Haiku scoring across the three evaluation datasets. Drop rate = pairs excluded from training entirely. SFT rows = pairs used for adapter training after 90/10 document-level train/val split.	39
3.2	SFT training rows per tier after 90/10 document-level train/val split. B3 trains on all pairs. QS-LoRA trains three separate adapters, one per tier.	41
3.3	Rank and scaling configuration per quality tier.	42
3.4	Training hyperparameters shared across all three tier-specific adapters. Rank and scaling (α) differ per tier as shown in Table 3.3.	43
3.5	Distilled local judge (v4) evaluation against Haiku on the OhioLine out-of-domain holdout ($n = 511$).	48
4.1	Summary of the three evaluation datasets. Generated pairs = total synthetic QA pairs produced before quality filtering. B3/B5 trains on all generated pairs (no filtering, after 90/10 train/val split). QS-LoRA trains on non-drop pairs only (high + medium + low tiers, after 90/10 split). Eval rows = human-authored QA pairs used for GA evaluation.	50

4.2	The three experimental conditions compared in this thesis. B3 and B5 train on all generated pairs including drop-tier pairs. QS-LoRA trains on non-drop pairs only, stratified by quality tier, and additionally requires a one-time Haiku quality scoring step per dataset.	52
4.3	Backbone models evaluated in the generalization study. QLoRA = 4-bit quantized LoRA used due to VRAM constraints. RepLiQA and Quoref results are available for all ten models; SQuAD v2 results are available for the 1B–14B models only.	57
5.1	RepLiQA GA scores (n=2,000) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). Δ = Ours – B3. Bold = best condition per model. $\dagger$ = CI excludes zero.	61
5.2	Quoref GA scores (n=2,418) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). $\dagger$ = CI excludes zero.	62
5.3	SQuAD v2 GA scores (n=11,873) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). $\dagger$ = CI excludes zero. Note: On SQuAD v2, B5 (AdaLoRA) ranks last among trained conditions; QS-LoRA outperforms both B3 and B5. Paired bootstrap: B5 vs B3 = -0.026 [$-0.046, -0.006$] $\dagger$ (B5 significantly worse than B3).	63
5.4	RepLiQA GA scores (n=2,000) for large backbone models (12B–70B). Bold = best condition per model. Llama-3.1-70B trained with 4-bit QLoRA on 4× A100-80GB.	65
5.5	Quoref GA scores (n=2,418) for large backbone models (12B–70B).	65
5.6	SQuAD v2 GA scores (n=11,873) for large backbone models. Llama-3.1-70B SQuAD v2 run not completed; those cells are omitted.	66
5.7	RepLiQA ceiling gap (Claude Opus 4.8, GA=4.40). Gap = ceiling – condition GA. % gain = (Ours – B3) / (ceiling – B3) × 100. Rows sorted by model size within family.	68
5.8	Quoref ceiling gap (Claude Opus 4.8, GA=4.45). Layout as Table 5.7.	68

5.9	Training cost comparison. B3 and B5 costs are scaled to reflect training on all generated pairs without quality filtering (the intended baseline configuration), proportional to row count. QS-LoRA (Ours) trains on non-drop pairs only by design. Scale factors: RepLiQA 1.22 $\times$, Quoref 2.42 $\times$, SQuAD 1.77 $\times$. AWS rate: \$5.12/hr per A100-SXM4-80GB. 70B uses 4 $\times$ GPU.	70
5.10	Mean Frobenius norm $\ \Delta W\ _F$ per target module (RepLiQA adapters). B5/Ours ratio = B5 Frobenius $\div$ Ours Frobenius. Values below 1.0 indicate B5 makes smaller weight updates than Ours.	72
5.11	Mean effective rank and rank@90% energy (RepLiQA adapters). Effective rank is computed from the entropy of the normalized singular value spectrum. Higher effective rank indicates energy is spread more evenly across singular directions.	72
5.12	Merge weight scheme ablation (Llama-3.2-3B reference). GA scores on RepLiQA ($n = 2,000$, Jul 9 re-audit), Quoref ($n = 2,418$), and SQuAD v2 ($n = 11,873$). $\dagger = 95\%$ CI excludes zero. Bold = best per dataset. Equal-rank: all three adapters at $r = 16$ (same as B3), merge weights 0.6/0.3/0.1.	76
5.13	Cross-model merge sweep on RepLiQA ($n = 2,000$). Tier = 0.6/0.3/0.1, Low-h = 0.4/0.4/0.2, Freq = frequency-weighted, Inv = 0.1/0.3/0.6, Equal = 1/1/1. Bold = best non-equal scheme per model. Δ = tier – best non-equal alternative (positive = tier wins).	78
5.14	Cross-model merge sweep on Quoref ($n = 2,418$). Same abbreviations as Table 5.13. Bold = best non-equal scheme.	79
5.15	Cross-model merge sweep on SQuAD v2 ($n = 11,873$). Same abbreviations as Table 5.13. Bold = best non-equal scheme. Gemma-3-12B freq/low-heavy/inverted cells not available.	80
5.16	Representative low-tier and drop-tier synthetic QA pairs from RepLiQA. Low-tier pairs are grounded (G=5) but irrelevant (R=1) – the answer does not address the question asked. Drop-tier pairs fail on grounding (G=1) – the answer contradicts or is unsupported by the document. Low-tier pairs carry domain vocabulary signal; drop-tier pairs are actively harmful as training signal.	81

5.17	High-only ablation (Llama-3.2-3B reference). Full QS-LoRA vs training on high-quality pairs only. $\Delta = \text{Ours} - \text{High-only}$. All Δ are positive, indicating that medium and low tier pairs contribute recoverable training signal.	81
5.18	Comparison of fine-tuned (FT) generator vs Haiku API generator on RepLiQA split 1. Usable pairs exclude drop-tier. Lexical coverage = token overlap between generated answer and reference answer. MiniLM cosine = semantic similarity via sentence embeddings.	82
5.19	Equal-rank ablation on Llama-3.2-3B (RepLiQA and Quoref). Equal-rank QS trains on quality-stratified tier subsets but uses $r = 16$ for all three adapters. This isolates the contribution of data stratification from differential rank allocation.	84
5.20	Distilled judge (v4, Llama-3.2-3B LoRA, 56,794 training pairs) evaluated against Haiku on per-domain held-out sets. OhioLine is out-of-domain (not seen in any training version). DROP is zero-shot (not in training data). False-high = pairs the student rates high but Haiku rates lower. False-drop = pairs the student drops but Haiku retains. . .	85
5.21	Pooled UAR (% , lower is better) across all three evaluation datasets. $\Delta = \text{Ours} - \text{baseline}$; negative = Ours better. Bold = lowest UAR per model. Ours beats B3 on all 10 models and B5 on 7/10.	88
5.22	Per-dataset UAR (%) for RepLiQA and Quoref. The Llama-3.2-3B RepLiQA result (-13.0 pp) and Llama-3.2-1B Quoref result (-10.4 pp) are the two strongest hallucination reductions in the study. Bold = lowest per model.	89
5.23	SQuAD v2 UAR (%). Llama-3.1-70B SQuAD run not completed. Bold = lowest per model. Ours beats B3 on all 9 evaluated models and B5 on 6/9.	90
5.24	Coverage and refusal behavior for Llama-3.2-3B on RepLiQA ($n_{\text{unanswerable}} = 396$, $n_{\text{answerable}} = 1,604$). Coverage = answered / total. Correct-refusal = abstained / unanswerable rows. Over-refusal = abstained / answerable rows.	90
5.25	Refusal win examples. Gold = “not found in document.” Ours correctly abstains (G=5) while B3 or B5 invents a specific unsupported fact (G=1). 92	

5.26	Answer win examples. Gold = correct grounded answer. Ours extracts the correct entity (G=5) while B3 or B5 invents the wrong entity or dumps unrelated context (G=1).	93
5.27	General capability scores (GA, 1–5) before and after QS-LoRA fine-tuning on RepLiQA. Evaluated on 100 questions across four categories. Lower fine-tuned score indicates capability degradation from domain-specific fine-tuning.	94

List of Figures

Figure		Page
1.1	Comparison of uniform LoRA (baseline) and QS-LoRA (proposed). Uniform LoRA trains a single adapter at fixed rank on all generated pairs, giving equal capacity to high- and low-quality examples. QS-LoRA scores each pair on four quality dimensions, assigns pairs to tiers, trains separate adapters at ranks proportional to quality ($r = 32, 16, 8$), and merges them into a single dense checkpoint. Both methods deploy as a single dense model with no inference overhead.	2
3.1	The QS-LoRA end-to-end pipeline. Starting from a private document collection, the pipeline proceeds through seven stages: document ingestion and chunking, synthetic QA generation, quality scoring by an LLM judge on four rubric dimensions (Grounding, Relevance, Document Necessity, Overall), tier assignment into high/medium/low quality bins based on judge score distribution, tier-specific LoRA adapter training at ranks proportional to quality ($r = 32, 16, 8$), offline tier-weighted dense merge ($\Delta W = 0.6A_H + 0.3A_M + 0.1A_L$), and on-premise deployment as a single dense checkpoint requiring no internet access, no retrieval index, and no adapter overhead at inference time.	33
5.1	Geometry of learned adapter updates across model scales (RepLiQA adapters, solid lines = Llama, dashed lines = Qwen). (a) Frobenius norm $\ \Delta W\ _F$ per LoRA module increases with scale for B3 and QS-LoRA while AdaLoRA’s update magnitude shrinks, with the magnitude hierarchy (B3 > QS-LoRA > AdaLoRA) persisting throughout. (b) Effective rank of ΔW : QS-LoRA maintains broad update directions at small scales while AdaLoRA concentrates capacity into fewer high-value directions as scale grows. The performance crossover at $\sim 12B$ aligns with the point where AdaLoRA’s targeted updates become better calibrated to large pretrained model requirements.	73

Chapter 1: Introduction

1.1 Motivation

Organizations increasingly need language model systems that can answer questions grounded in their own internal documents: enterprise knowledge bases, technical manuals, legal archives, regulatory filings, and operational records. These document collections are private by nature, unavailable for pretraining, and often too sensitive to transmit to external API providers. The practical question is how to build a language model that internalizes the content of such a collection at acceptable cost and without exposing the documents to third-party infrastructure. This thesis proposes Quality-Stratified Low-Rank Adaptation (QS-LoRA), a parameter-efficient fine-tuning method that addresses this problem by allocating greater model capacity to higher-quality synthetic training pairs rather than treating all pairs uniformly or discarding lower-quality ones entirely. The method is designed for the 1B–8B parameter range and consistently outperforms uniform LoRA across ten backbone models, three model families, and three document-grounded question-answering benchmarks, with a known limitation at 12B and above where adaptive rank allocation methods are more appropriate. Two strategies have emerged for grounding language models in document content: retrieval-augmented generation (RAG) and parameter-efficient fine-tuning.

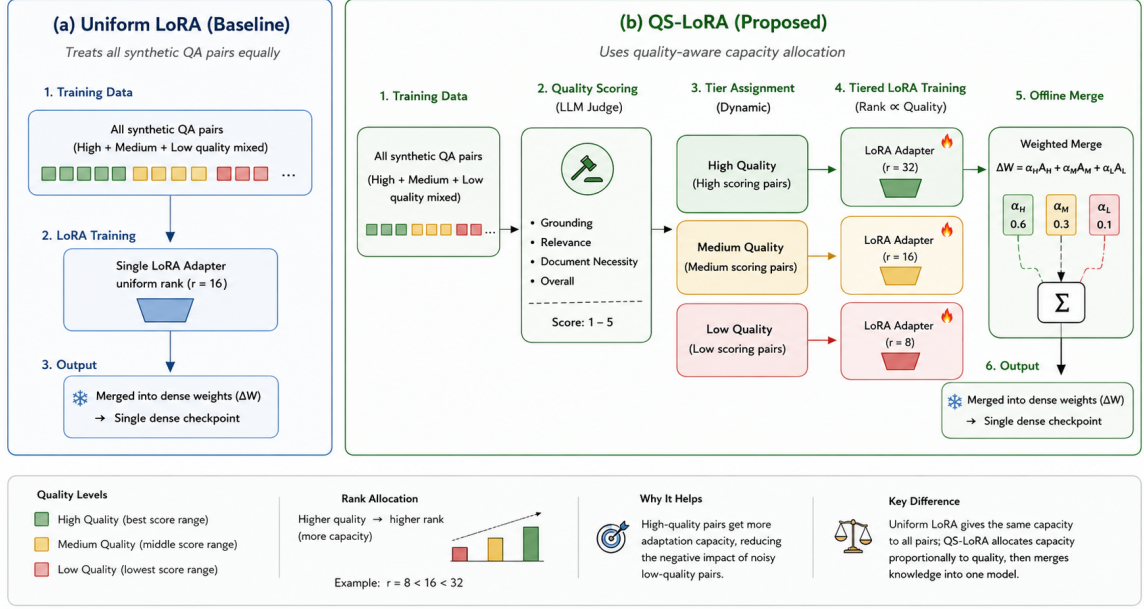


Figure 1.1: Comparison of uniform LoRA (baseline) and QS-LoRA (proposed). Uniform LoRA trains a single adapter at fixed rank on all generated pairs, giving equal capacity to high- and low-quality examples. QS-LoRA scores each pair on four quality dimensions, assigns pairs to tiers, trains separate adapters at ranks proportional to quality ($r = 32, 16, 8$), and merges them into a single dense checkpoint. Both methods deploy as a single dense model with no inference overhead.

RAG pipelines retrieve relevant passages from an indexed document store at inference time and condition the model on those passages alongside the user query. Fine-tuning internalizes document content directly into model weights through supervised training on question-answer pairs derived from the documents. When source documents are available at inference time, RAG has been found to outperform fine-tuning in Ovadia et al.’s systematic comparison on held-out question answering benchmarks [49]. However, RAG requires maintaining an indexed document store, introduces retrieval latency at every inference step, and degrades when the relevant passage is not retrieved.

For settings where low-latency, offline, or air-gapped deployment is a requirement, fine-tuning is the more practical choice.

The case for fine-tuning is further strengthened by recent advances in small open-weight language models. Model families such as Llama [61], Qwen [2], and Gemma [19] now offer capable instruction-following models at parameter scales from one billion to fourteen billion, which can be fine-tuned on a single commodity GPU in a matter of hours. Table 1.1 illustrates the practical cost difference across model scales, derived from actual training wall times measured in this thesis on AWS-equivalent A100-80GB hardware.

Table 1.1: Actual training cost for one fine-tuning run (QS-LoRA, RepLiQA dataset) on AWS-equivalent A100-SXM4-80GB hardware at \$5.12/hr per GPU. Times measured empirically in this thesis. 70B requires $4\times$ A100 GPUs; all other models use $1\times$ GPU.

Model	Parameters	GPUs	Wall Time	AWS Cost
Llama-3.2-1B	1B	1	1h 28m	\$7.54
Llama-3.2-3B	3B	1	2h 38m	\$13.45
Llama-3.1-8B	8B	1	4h 07m	\$21.04
Qwen2.5-14B	14B	1	8h 54m	\$45.52
Gemma-3-12B	12B	1	10h 26m	\$53.39
Llama-3.1-70B	70B	4	32h 40m	\$669.07

The cost difference between a 3B and a 70B fine-tuning run is approximately $50\times$. A single 70B training run costs \$669 and requires four GPUs running for 33 hours, hardware unavailable to most individual practitioners or small organizations. A 3B model costs \$13 per run on a single GPU and completes in under three hours. For use cases involving periodic re-fine-tuning as documents are updated, the 1B–8B scale

range is the practical deployment target for the vast majority of organizations without large-scale GPU infrastructure.

The critical question, then, is whether a small fine-tuned model can be made to perform well enough on private document question answering to serve as a useful alternative to large models or retrieval-augmented systems. This thesis addresses one specific bottleneck that limits small model fine-tuning quality: the noise in the synthetic data used to train these models.

1.1.1 Scope of Claims

This thesis proposes and validates QS-LoRA: a quality-aware fine-tuning pipeline that consistently outperforms uniform LoRA and AdaLoRA (at 1B–8B) across ten backbone models and three document-grounded benchmarks, using a single fixed random seed per condition. Three limitations bound the scope. First, the method is validated in the 1B–8B range – at 12B and above, AdaLoRA is the preferred method. Second, the design simultaneously changes data partitioning, rank allocation, and merge weighting; the ablations only partially separate their contributions. Third, all results reflect judge-based evaluation rather than human expert assessment, and do not capture training variance across seeds. The contribution is a well-performing, end-to-end validated design for private-document fine-tuning, not a new general PEFT principle..

In plain terms, this thesis makes two concrete points:

- **Design:** Sorting synthetic training pairs by quality, giving better pairs more model capacity, and combining the results into one model is a principled and practical approach to private-document fine-tuning.

- **Empirical result:** This configuration consistently outperforms uniform LoRA and AdaLoRA at 1B–8B across ten backbone models and three benchmarks in single-seed experiments.

1.2 Problem Statement

Private document collections have no associated human-annotated question-answer datasets. The only scalable path to building fine-tuning supervision is to generate synthetic question-answer pairs from the documents using a language model. This generation process is imperfect. Yue et al. [69] showed that synthetic QA pipelines consistently produce a non-trivial fraction of noisy pairs, including questions whose answers do not require the source document (trivial questions) and questions that are semantically mismatched from their generated answers. In this thesis, the proportion of pairs discarded by the quality scoring step ranges from 30% to 54% across the three synthetic training pools (RepLiQA, Quoref, and SQuAD v2; see Table 3.1 and Table 4.1), this range reflects variation across document types and is distinct from the discard rates reported by Yue et al.

The standard response to this noise is hard filtering: pairs below a quality threshold are discarded entirely, and the surviving pairs are used to train a single LoRA adapter with uniform rank. This approach has two limitations. First, filtering reduces training data volume, which is already limited in the private document setting. Second, it treats all surviving pairs as equally informative, regardless of quality variation within the accepted set. A high-quality pair—one that is well-grounded, on-topic, and requires genuine comprehension of the source document—is given the same model capacity as a medium-quality pair that is mostly correct but exhibits one weak rubric dimension.

The model cannot differentiate its parameter updates by the quality of the training signal.

This thesis addresses the following problem: *given a synthetic question-answer pool of heterogeneous quality derived from a private document collection, how can a parameter-efficient fine-tuning method be designed to allocate greater model capacity to higher-quality pairs without discarding lower-quality pairs entirely?*

The assumption that lower-quality pairs carry recoverable signal, rather than being pure noise, is empirically supported by an ablation experiment reported in Chapter 5: training on high-quality pairs only degrades performance by 0.02–0.12 GA points across three datasets relative to the full method, confirming that medium and low quality pairs contribute to fine-tuning outcomes when assigned appropriately reduced model capacity.

1.3 Research Objectives

The thesis addresses the following five research objectives:

1. **Quality-stratified fine-tuning.** Does combining quality-scored data partitioning, differential rank allocation, and tier-weighted merging into a single pipeline outperform uniform LoRA and AdaLoRA on document-grounded QA?

Sub-question: Do medium and low quality pairs contribute recoverable signal when retained at reduced rank, or is the gain driven entirely by the high-quality tier?

2. **End-to-end pipeline for private document deployment.** Can a complete, reproducible pipeline be constructed that takes a private document collection as input and produces a deployed dense model checkpoint as output, without

requiring any human annotation or access to external API providers at inference time?

3. **Scale boundary identification.** At what parameter scale does QS-LoRA’s advantage over AdaLoRA diminish or reverse, and can the transition be explained mechanistically through weight update geometry?
4. **Air-gapped judge replacement.** Can a small locally-hosted model fully replace Claude Haiku 4.5 for quality scoring in this pipeline, enabling fully on-premise execution without any external API calls or per-query cost?
5. **Inference efficiency.** Does the offline dense merge of tier-specific adapters eliminate runtime adapter overhead relative to serving LoRA dynamically, and is the resulting throughput improvement sufficient for latency-sensitive on-premise deployment?

1.4 Contributions

This thesis makes the following contributions, listed in order of significance:

1. **Quality-Stratified Low-Rank Adaptation (QS-LoRA).** A fine-tuning method that assigns each synthetic QA pair to one of three quality tiers using a multi-dimensional LLM judge rubric, trains a separate LoRA adapter per tier at rank $r \in \{32, 16, 8\}$ proportional to tier quality, and merges the three adapters offline into a single dense checkpoint using tier-weighted coefficients. Evaluated across 29 model-dataset combinations, QS-LoRA outperforms uniform LoRA in every cell (mean +0.14 GA) and outperforms both baselines in all 21

small-model cells (1B–8B). At 12B and above, AdaLoRA is preferred, this is the primary scope limitation of the method.

2. **End-to-end private document fine-tuning pipeline.** A complete, reproducible system pipeline that takes a private document collection as input and produces a deployed dense model checkpoint as output. The pipeline covers document ingestion and chunking, synthetic QA generation using a fine-tuned small generator model, quality scoring and tier assignment using a multi-axis rubric judge, tier-specific LoRA adapter training, offline tier-weighted dense merge, and evaluation against human-authored reference answers. The pipeline requires no human annotation, operates entirely on-premise, and transmits no document content to external providers at inference time. The offline dense merge delivers $2.3\text{--}3.2\times$ faster inference throughput than serving a LoRA adapter at runtime.
3. **Cross-model generalization study.** A systematic evaluation across ten backbone models spanning three model families (Llama, Qwen2.5, Gemma-3) at 1B–70B, totaling 29 evaluated model-dataset combinations. The study identifies the scale boundary at 12B and above where AdaLoRA becomes preferable, and provides a mechanistic explanation through SVD analysis of adapter weight update geometry.
4. **Distilled local quality judge.** A Llama-3.2-3B LoRA adapter trained on 56,794 Haiku-scored synthetic QA pairs to replicate the Bedrock Haiku quality scoring step locally, enabling fully air-gapped pipeline execution without per-call API cost. The distilled judge achieves 78.5% tier agreement, Cohen’s $\kappa = 0.62$,

and Spearman $\rho = 0.79$ against Haiku scores on an out-of-domain holdout of 511 pairs, demonstrating that the quality scoring step can be internalized into a small local model without significant degradation in tier assignment accuracy.

5. **Training cost and inference efficiency.** QS-LoRA is modestly cheaper than uniform LoRA overall (see Table 5.9), but the primary efficiency contribution is methodological rather than economic: the offline dense merge eliminates runtime adapter overhead entirely, delivering 2.3–3.2 $\times$ faster inference throughput than serving a LoRA adapter dynamically, a practically significant benefit for latency-sensitive on-premise deployment.

The novelty of this work lies specifically in the combination of three components in a private-document fine-tuning setting: (1) LLM-judge quality scoring of synthetic training pairs on a multi-dimensional rubric, (2) tiered rank allocation proportional to quality, and (3) offline tier-weighted dense merging into a single deployable checkpoint. No single component is novel in isolation, quality scoring, low-rank adaptation, and model merging each exist in prior work. The contribution is their integration and empirical validation as a complete pipeline for the constrained on-premise private-document deployment setting. This thesis does not claim a new general principle of parameter-efficient fine-tuning, and the results should not be interpreted as evidence that quality stratification improves PEFT broadly, only that this specific configuration outperforms the evaluated baselines in this specific setting.

1.5 Thesis Organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews the technical literature underlying this work, covering large language model fine-tuning, parameter-efficient fine-tuning methods, low-rank adaptation and its variants, retrieval-augmented generation, synthetic question-answer generation, LLM-as-judge evaluation, and adapter merging.

Chapter 3 presents the QS-LoRA method in full. It describes the complete end-to-end pipeline from document ingestion through deployment, covering the synthetic QA generation step, the quality scoring rubric and tier assignment rules, the tier-specific LoRA adapter training procedure, the offline tier-weighted dense merge step, and the distilled local judge that enables air-gapped execution of the quality scoring step.

Chapter 4 describes the experimental setup, including the three evaluation datasets (RepLiQA, Quoref, SQuAD v2), the five baseline conditions, the hardware configuration, training hyperparameters, and the Gold Alignment evaluation metric and judge rubric.

Chapter 5 reports the experimental results. It presents the main comparison of QS-LoRA against uniform LoRA and AdaLoRA on the reference Llama-3.2-3B backbone, the ablation study on merge strategy and low-quality pair contribution, the cross-model generalization study across ten backbone models including Llama-3.1-70B, the SVD adapter analysis providing mechanistic explanation for the scale-dependent behavior, and the training cost and inference latency analysis.

Chapter 6 summarizes the contributions of the thesis, discusses the limitations of the proposed method, and identifies directions for future work.

Chapter 2: Background and Related Work

This chapter reviews the technical literature that underpins the problem of fine-tuning large language models on private document collections. The areas covered are organized as follows. Section 2.1 introduces the foundations of large language model fine-tuning and the key challenges that motivate parameter-efficient approaches. Section 2.2 surveys the parameter-efficient fine-tuning literature. Section 2.3 reviews low-rank adaptation in detail, including its principal variants and extensions. Section 2.4 covers retrieval-augmented generation as a complementary and competing strategy for knowledge injection. Section 2.5 surveys methods for synthetic question-answer generation from document collections. Section 2.6 reviews the LLM-as-judge methodology for automated evaluation and quality scoring. Section 2.7 covers the literature on merging multiple fine-tuned adapters into a single model.

2.1 Fine-Tuning of Large Language Models

Large language models (LLMs) such as GPT [4], LLaMA [61], and Qwen [2] are pretrained on massive corpora using a next-token prediction objective. Pretraining instills general language understanding, factual knowledge, and reasoning patterns across diverse domains. However, pretrained models are not directly optimized for

instruction following, task-specific response formatting, or narrow domain expertise. Fine-tuning is the standard method for adapting pretrained models to these downstream requirements.

2.1.1 Full Fine-Tuning

In full fine-tuning, all model parameters are updated on a task-specific dataset by minimizing a supervised loss, typically cross-entropy over target token sequences. Early work established that even a small amount of task-specific gradient signal applied to a large pretrained model produces substantial improvements over zero-shot inference [64]. Full fine-tuning therefore remains the theoretical upper bound against which more constrained adaptation methods are measured.

The practical limitation of full fine-tuning is computational cost. A model with billions of parameters requires a correspondingly large optimizer state, gradient buffers, and activation memory during training. For a model with N parameters trained with the Adam optimizer [30], the optimizer state alone requires $2N$ additional parameters in memory. At the scale of modern LLMs, this requirement frequently exceeds the capacity of a single GPU, necessitating model parallelism, gradient checkpointing, or quantized training strategies that add system complexity and reduce iteration speed.

A further concern with full fine-tuning is catastrophic forgetting [45], where updating all parameters on a narrow dataset can overwrite broadly useful representations learned during pretraining. This is especially problematic in domain-specific fine-tuning where training data is limited in volume and diversity, as the model may lose general instruction-following capabilities while gaining narrow task performance.

2.1.2 Instruction Tuning

Instruction tuning refers to fine-tuning a pretrained language model on a dataset of instruction-response pairs so that the model learns to follow natural language directives rather than simply complete arbitrary text. Introduced at scale by Wei et al. [64] and further developed in the FLAN [10] and InstructGPT [48] lines of work, instruction tuning significantly improves a model’s ability to produce appropriate responses across a wide variety of tasks without task-specific prompt engineering.

The dataset composition and quality used for instruction tuning have a large effect on the resulting model. Zhou et al. [74] demonstrated in the LIMA study that training on as few as 1,000 carefully selected high-quality instruction-response pairs can match the performance of models trained on far larger but lower-quality instruction corpora. This finding highlighted that data quality exerts more influence on final behavior than data volume across a wide range of instruction-tuning settings. Subsequent work formalized this observation into automatic data selection criteria, including LLM-based quality scoring [5], difficulty-based selection [36], multi-axis scoring with diversity [42], and gradient-based influence estimation [66].

2.1.3 Domain-Specific Fine-Tuning

Domain-specific fine-tuning adapts a pretrained model to a specialized corpus such as medical records, legal documents, scientific literature, or enterprise knowledge bases. The goal is to make the model more accurate and fluent within the target domain without losing its general language capabilities. Domain-specific fine-tuning differs from general instruction tuning in that the training signal is typically derived from domain text rather than from human-curated task demonstrations.

A persistent challenge in domain-specific fine-tuning is the availability of high-quality training data. Human-annotated question-answer pairs are expensive to produce at scale, especially for proprietary or confidential document collections. This has motivated the use of synthetic data generation pipelines in which an LLM is used to automatically produce question-answer pairs from document text [63, 51]. The quality and diversity of such synthetic data has a direct effect on fine-tuning outcomes, since the model cannot easily distinguish genuine comprehension questions from superficial or malformed ones if they are presented with equal weight during training. What is missing from the existing fine-tuning literature is a method that goes beyond treating all synthetic pairs equally or discarding low-quality ones entirely, instead allocating model capacity in proportion to the quality of each training pair. This thesis addresses that gap.

2.2 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) methods reduce the computational cost of adaptation by introducing a small number of trainable parameters while keeping the pretrained model weights frozen. The objective is to match or closely approximate the performance of full fine-tuning at a fraction of the memory and compute cost. PEFT methods have become the dominant practical approach for adapting large language models in settings where full fine-tuning is infeasible.

2.2.1 Adapter Layers

Adapter methods [23] insert small trainable modules between the layers of a frozen pretrained model. Each adapter typically consists of a down-projection, a nonlinearity, and an up-projection, forming a bottleneck that introduces a small

number of additional parameters per layer. During fine-tuning, only the adapter parameters are updated. At inference time, the adapter outputs are combined with the frozen residual stream of the pretrained model.

Adapters were initially developed for transformer-based natural language understanding models [14] and have since been extended to generative models and vision transformers. Their key advantage is modularity: a single pretrained backbone can be paired with task-specific adapters, allowing multiple tasks to share the same base model without parameter conflicts. Their primary limitation is the introduction of additional inference latency, since the adapter forward pass adds computation to every layer of the model.

2.2.2 Prefix Tuning and Prompt Tuning

Prefix tuning [38] prepends a sequence of trainable continuous vectors to the input of each transformer layer. These prefix vectors act as soft prompts that condition the model’s attention and feed-forward computations without modifying the underlying weights. The trainable prefix is optimized during fine-tuning, and the pretrained model weights remain frozen throughout. Prefix tuning demonstrated that competitive task performance could be achieved with fewer than 0.1% of the pretrained model’s parameters.

Prompt tuning [33] simplified this approach by applying soft prompt vectors only at the input embedding layer rather than at every transformer layer, further reducing the number of trainable parameters. These methods collectively established that the input conditioning space of a transformer provides sufficient degrees of freedom to adapt model behavior across a wide variety of tasks without any weight modification.

2.2.3 Sparse and Selective Fine-Tuning

An alternative to adding new parameters is to select a subset of the existing pretrained parameters for updating while keeping the remainder frozen. Diff pruning [20] learns a sparse difference vector over the full parameter space and applies L0 regularization to encourage most entries to remain zero. BitFit [70] takes a simpler approach, updating only the bias terms of a pretrained model and leaving all weight matrices frozen. These selective approaches demonstrate that task-relevant adaptation can often be captured by a small subset of existing parameters, though they offer less flexibility than methods that explicitly introduce new trainable structure. All of the methods reviewed in this section allocate the same capacity to every training example, regardless of its quality. This thesis introduces a departure from that assumption.

2.3 Low-Rank Adaptation and Its Variants

Low-rank adaptation (LoRA) [24] is currently the most widely adopted parameter-efficient fine-tuning method for large language models. Its core idea is to constrain the weight updates introduced by fine-tuning to a low-rank subspace of the original parameter space. This approximation is motivated by the observation, noted in earlier work on intrinsic dimensionality [35, 1], that the effective degrees of freedom required to adapt a pretrained model to a target task are much smaller than the full parameter count would suggest.

2.3.1 LoRA: Low-Rank Adaptation

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA represents the fine-tuning update as the product of two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k). \quad (2.1)$$

The pretrained weights W_0 are frozen throughout training. Only A and B are trainable. A is initialized from a random Gaussian distribution and B is initialized to zero, so $\Delta W = 0$ at the start of fine-tuning and training begins from the pretrained model’s behavior. At inference time, the product BA is merged into W_0 so that the adapted model introduces no additional latency compared with the base model.

The rank r is the primary hyperparameter governing the capacity of the adaptation. Smaller values of r reduce the number of trainable parameters and act as a stronger regularizer, while larger values allow more expressive updates at the cost of a larger parameter budget. Hu et al. [24] showed that a rank as low as $r = 4$ or $r = 8$ is sufficient to match or approach full fine-tuning performance on a variety of natural language understanding and generation tasks, suggesting that the intrinsic dimensionality of fine-tuning is indeed much lower than the ambient parameter space.

LoRA is typically applied to the query and value projection matrices of self-attention layers, though it can also be extended to the key projections, feed-forward layers, and other linear transformations within a transformer block.

2.3.2 AdaLoRA: Adaptive Budget Allocation

A limitation of standard LoRA is that it assigns the same rank r to all adapted weight matrices regardless of their individual importance to the downstream task. This

uniform assignment may be suboptimal because different layers and different weight matrices within a layer are known to contribute unequally to task performance [46, 62].

AdaLoRA [71] addresses this by parameterizing the weight update through a singular value decomposition:

$$\Delta W = P\Lambda Q, \quad P \in \mathbb{R}^{d \times r}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_r), \quad Q \in \mathbb{R}^{r \times k}. \quad (2.2)$$

During training, AdaLoRA maintains an importance score for each singular value based on the magnitude of its gradient contribution to the task loss. Singular values with low importance scores are progressively pruned, effectively reducing the rank of less important weight updates while preserving capacity in more sensitive layers. A global parameter budget is enforced across all weight matrices, so rank is redistributed from less important to more important components rather than simply discarded.

AdaLoRA consistently outperforms uniform LoRA at equivalent parameter budgets across a range of natural language processing benchmarks, demonstrating that adaptive rank allocation is a more efficient use of a fixed parameter budget than uniform assignment.

2.3.3 QLoRA: Quantized Low-Rank Adaptation

QLoRA [13] extends LoRA to very large models by combining 4-bit quantization of the pretrained weights with LoRA adapters trained in a higher precision. The pretrained model is loaded in 4-bit NormalFloat (NF4) format using a double quantization strategy that further reduces memory overhead. LoRA adapters are then applied on top of the quantized base model and trained in bfloat16, so that gradient flow passes through the dequantized weights during backpropagation.

QLoRA demonstrated that a 65-billion-parameter language model could be fine-tuned on a single 48 GB GPU with performance competitive with full 16-bit fine-tuning. This result dramatically expanded the range of model scales accessible to PEFT practitioners with limited hardware, making fine-tuning of very large models practical without access to multi-GPU clusters.

2.3.4 DoRA: Weight-Decomposed Low-Rank Adaptation

DoRA [41] decomposes the pretrained weight matrix into a magnitude component and a direction component, and applies LoRA exclusively to the directional update while learning the magnitude separately. This decomposition is motivated by an analysis of the learning patterns of full fine-tuning versus standard LoRA, which shows that LoRA tends to update magnitude and direction jointly in a manner that differs from full fine-tuning’s characteristic pattern of proportionally larger directional updates.

By separating these two components, DoRA consistently improves over standard LoRA across a variety of fine-tuning benchmarks while maintaining LoRA’s inference efficiency advantage of zero additional latency after weight merging.

2.3.5 Other LoRA Extensions

Several additional extensions of LoRA have been proposed to address its remaining limitations. LoRA+ [22] corrects for an imbalance in the optimal learning rates for the A and B matrices, showing that B should be updated with a higher learning rate than A for optimal convergence, and derives this result from feature learning theory. VeRA [31] further reduces trainable parameter count by sharing the A and B

matrices across all layers and training only small per-layer scaling vectors, achieving competitive performance at a fraction of LoRA’s already reduced parameter count.

LongLoRA [7] extends LoRA to long-context fine-tuning by combining shifted sparse attention during training with standard LoRA adapters, enabling efficient fine-tuning of models on sequences far longer than those seen during pretraining. ElaLoRA [72] extends AdaLoRA with an expansion mechanism that both prunes and adds singular components during training, allowing rank to increase in highly sensitive layers rather than only decrease.

TsqLoRA [6] represents a recent direction that integrates data quality into the adapter training process. It introduces a quality-aware sampling mechanism that assigns higher selection probability to training examples estimated to be more informative, and pairs this with a sensitivity-aware dynamic rank allocation module that adjusts per-layer rank based on gradient sensitivity during training. TsqLoRA is evaluated on GLUE classification benchmarks and is the first published method to explicitly combine a data quality signal with per-layer rank adaptation in a single training run. However, TsqLoRA applies quality as a sampling mechanism on the parameter axis (per layer), while this thesis applies quality as a partitioning mechanism on the data axis (per tier). These are orthogonal approaches, and the private document fine-tuning setting, with its heterogeneous synthetic data and air-gapped deployment requirements, is not addressed by any of the methods reviewed above.

2.4 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [34] is a complementary paradigm to fine-tuning for grounding language model responses in external knowledge. Rather than

storing knowledge in model weights through training, RAG pipelines retrieve relevant passages from an indexed document collection at inference time and condition the generator on those passages alongside the user query.

2.4.1 Architecture and Core Components

A standard RAG pipeline consists of three components: a document corpus indexed for retrieval, a retriever that identifies relevant passages given a query, and a generator that produces a response conditioned on both the query and the retrieved context. The retriever may be sparse, using term-based matching such as BM25 [53], or dense, using learned query and passage encoders such as DPR [28]. Dense retrievers encode queries and passages into a shared embedding space and identify relevant passages by approximate nearest-neighbor search over the passage index.

More recent retrieval models such as ColBERT [29] use late interaction between query and passage token representations rather than a single aggregated vector per passage, improving retrieval quality at the cost of increased index storage. Hybrid retrieval systems combine sparse and dense signals to balance the recall advantages of BM25 with the semantic generalization of dense encoders.

2.4.2 Limitations of Retrieval-Augmented Generation

Despite strong performance on open-domain question answering benchmarks, RAG has several documented limitations. Retrieval failures, where the relevant passage is not present in the top- k results, directly degrade generation quality because the generator receives no useful context [57]. Long-context sensitivity is a related concern: when multiple retrieved passages are concatenated, models tend to attend disproportionately to passages near the beginning or end of the context window, a phenomenon known

as the lost-in-the-middle effect [40]. RAG also introduces inference latency due to the retrieval step, requires maintaining and updating a passage index as documents change, and cannot easily answer questions that require synthesizing information distributed across many documents rather than extracting a localized passage.

2.4.3 Fine-Tuning vs. Retrieval-Augmented Generation

The tradeoff between fine-tuning and RAG for knowledge-intensive tasks has been studied empirically in several settings. Ovadia et al. [49] conducted a systematic comparison of fine-tuning and retrieval-based knowledge injection across multiple question-answering benchmarks, finding that RAG has been found to outperform fine-tuning in Ovadia et al.’s systematic comparison when documents are available at inference time, while fine-tuning generalizes better when retrieval is absent or unreliable. Soudani et al. [58] studied the performance gap for less common knowledge domains and found that fine-tuning on domain-specific synthetic data can partially close but not fully eliminate the advantage of RAG with access to the source documents.

More recent work on synthetic mixed training [21] demonstrated that combining synthetic question-answer pairs with synthetic documents during fine-tuning can close the gap to RAG by a larger margin, with the key insight that synthetic question-answer pairs primarily teach behavioral knowledge transferable across domains, while synthetic documents primarily teach domain-specific factual knowledge. Despite these advances, RAG with direct access to the source documents at inference time remains a difficult upper bound for any fine-tuning method to exceed on document-grounded question answering.

2.5 Synthetic Question-Answer Generation

The use of language models to generate synthetic training data has become a central technique in settings where human-annotated supervision is scarce or expensive. For document-grounded question answering specifically, synthetic generation provides the only scalable path to building fine-tuning datasets from private or proprietary document collections.

2.5.1 Self-Supervised and Model-Generated Instruction Data

Self-Instruct [63] established the feasibility of using a language model to bootstrap its own instruction-following training data. Starting from a small seed of human-written instruction-response pairs, the method iteratively prompts the model to generate new instructions and responses, filters the outputs by quality heuristics, and adds the accepted pairs to the training pool. This process demonstrated that a model could substantially improve its own instruction-following capability through self-generated data without requiring large amounts of human annotation.

Alpaca [60] applied the Self-Instruct methodology at larger scale using GPT-3.5 (text-davinci-003) as the generator, producing 52,000 instruction-following pairs from 175 human-written seed examples and fine-tuning LLaMA-7B on the resulting dataset. This established a widely-used recipe for instruction data generation that subsequent work has refined through improved generator models, quality filtering, and more structured generation prompts.

Instruction backtranslation [37] took an alternative approach by generating instructions that would naturally elicit a given passage from a web corpus rather than generating responses from instructions. This inverted pipeline avoids the need for a

strong instruction-following generator and instead leverages the breadth of existing web text as a source of naturally occurring responses.

2.5.2 Document-Grounded Question-Answer Generation

Generating question-answer pairs specifically grounded in document content requires the generator to produce questions whose answers are contained in or inferable from a source passage, rather than questions drawn from the model’s parametric knowledge. Early approaches used templated extraction patterns to generate factoid questions from named entities and their surrounding context [15]. More recent methods use instruction-tuned LLMs prompted with the source passage and asked to generate questions that test comprehension of that passage.

Source2Synth [43] extended this to multi-hop question generation from structured data sources, showing that synthetic multi-hop question-answer pairs can improve downstream retrieval-augmented generation performance on complex reasoning benchmarks. GraphGen [8] constructed knowledge-graph-structured question-answer pairs from document collections, enabling more systematic coverage of entities and their relationships.

A shared challenge in document-grounded generation is that simple chunk-level prompting tends to produce lookup-style questions rather than reasoning-intensive ones. The complexity of the generated question distribution affects the quality signal available to any downstream quality scoring step: if all generated questions are surface-level retrievals, the quality variance across pairs is low and quality scores provide a weaker partitioning signal.

2.5.3 Quality Filtering of Synthetic Data

Not all synthetically generated question-answer pairs are equally useful for fine-tuning. Common failure modes include malformed questions, answers that are inconsistent with the source passage, questions that are answerable without the passage, and questions that test only superficial lexical overlap rather than comprehension. Several methods have been proposed to filter these low-quality pairs before training.

AlpaGasus [5] used GPT-4 as a quality judge to score each Alpaca pair on a five-point scale and retained only pairs scoring above a threshold, reducing the training set from 52,000 to approximately 9,000 pairs while improving performance on multiple benchmarks. RAGAS-based filtering [18] applied a retrieval-augmented quality assessment framework specifically to document-grounded synthetic pairs, scoring faithfulness, relevance, and coherence before filtering. These methods share the common design choice of treating low-quality pairs as noise to be discarded rather than as signal to be retained at a reduced weight.

2.6 LLM-as-Judge for Automated Evaluation and Quality Scoring

The use of large language models as automated evaluators has become widespread in natural language generation research, both for evaluating final model outputs and for scoring training data quality. This section reviews the methodological foundations of LLM-as-judge approaches and the known biases that must be considered when interpreting their outputs.

2.6.1 Foundations of LLM-Based Evaluation

Zheng et al. [73] conducted a systematic study of LLM-based evaluation using MT-Bench, a set of multi-turn conversation benchmarks, and found that GPT-4 as a judge achieves agreement with human expert annotations at rates comparable to inter-human agreement. This result validated the use of LLM judges as a scalable substitute for human evaluation on instruction-following tasks where reference answers are unavailable or where response quality is multidimensional.

LLM-as-judge frameworks have since been widely adopted in instruction tuning research [5, 42], safety evaluation [3], and synthetic data quality scoring [18]. Common evaluation protocols include pairwise comparison, where the judge selects the better of two responses, and pointwise scoring, where the judge assigns an absolute score to a single response according to a rubric.

2.6.2 Known Biases in LLM Evaluation

A growing body of work has documented systematic biases in LLM-based evaluation that can distort scores in ways unrelated to actual response quality. Understanding these biases is important for any system that uses LLM judges for quality scoring or model evaluation.

Self-preference bias. Panickssery et al. [50] showed that LLMs preferentially score outputs generated by models from the same family more highly than outputs of equal quality from other model families, even when the judge has no explicit information about the source of each response. This bias means that using a judge from the same model family as the evaluated system can systematically inflate scores relative to an independent judge.

Verbosity bias. Saito et al. [55] documented that LLM judges consistently assign higher scores to longer, more elaborate responses even when the additional length does not contribute additional factual content. This creates an incentive for generation systems to produce verbose outputs rather than concise and accurate ones, which can mislead quality scoring when length is not explicitly controlled.

Position bias. Shi et al. [56] identified a significant position bias in pairwise LLM evaluation, where judges disproportionately prefer the response presented in the first position regardless of actual quality. This bias affects any evaluation protocol that presents responses in a fixed order and can be partially mitigated by evaluating each pair in both orderings and averaging.

Format bias. Several studies have noted that LLM judges assign higher scores to responses with structured formatting such as bullet points, numbered lists, and markdown headers even when the same information presented in plain prose would be equally or more informative [73]. This introduces a confound between formatting style and judged quality.

2.6.3 Rubric-Based Pointwise Scoring

Rubric-based pointwise scoring presents the judge with an explicit evaluation rubric that defines specific criteria and anchors the scoring scale to concrete descriptions of each quality level. This approach is more structured than open-ended pairwise comparison and can reduce variability by constraining the judge’s evaluation to predefined axes such as factual accuracy, completeness, relevance, and fluency. Rubric-based protocols have been used in instruction data curation [5], safety annotation [3], and retrieval-augmented generation evaluation [18].

A key advantage of rubric-based scoring over open-ended evaluation is that the rubric makes the quality criteria explicit and interpretable. The judge’s score can be audited against the criteria, and the rubric can be revised to address specific failure modes as they are identified. This makes rubric-based scoring more transparent and correctable than holistic judgments.

2.7 Merging of Fine-Tuned Adapters

Model merging refers to the construction of a single model from two or more independently fine-tuned models without access to the original training data or further gradient-based optimization. Merging has been studied in the context of both full fine-tuned models and lightweight adapters such as LoRA, with distinct challenges arising in each setting.

2.7.1 Parameter-Space Merging of Full Models

Wortsman et al. [65] introduced the model soup paradigm, showing that averaging the parameters of multiple models fine-tuned from the same pretrained initialization with different hyperparameter configurations produces a single model that outperforms any individual member of the ensemble on held-out evaluation. This result established that independently fine-tuned models from a common initialization can be combined in parameter space rather than through ensembled inference, eliminating the inference-time cost of running multiple models.

Fisher Merging [44] extended this idea by weighting each model’s parameters according to the diagonal of its empirical Fisher information matrix, approximating the importance of each parameter to that model’s task. This principled weighting

scheme outperforms simple uniform averaging when the merged models have been fine-tuned on different tasks and their parameter distributions differ meaningfully.

RegMean [27] formulated merging as a regression problem: given the activations of each model on a small set of unlabeled inputs, find a merged weight matrix that minimizes the sum of squared differences between the merged model’s activations and those of the individual models. This approach is more computationally demanding than averaging but produces more accurate merges on diverse task combinations.

2.7.2 Interference in Multi-Task Merging

A central challenge in model merging is parameter interference: when two models have been fine-tuned on different tasks, their parameter updates may conflict in directions that partially cancel one another when linearly combined. Yadav et al. [67] analyzed this phenomenon through the lens of sign conflicts in parameter delta vectors and introduced TIES-Merging, which resolves conflicts by retaining only parameters whose signs are consistent across a majority of the merged models and trimming parameters with small absolute magnitude before merging. Empirically, TIES-Merging outperforms simple averaging across a range of multi-task merging benchmarks.

DARE [68] introduced a complementary approach based on random parameter sparsification: before merging, each model’s parameter delta is randomly pruned at a high rate (e.g., 90% of entries set to zero) and the surviving entries are rescaled to preserve the expected magnitude. DARE reduces interference by ensuring that the merged model’s parameter space is dominated by the most salient updates from each constituent model rather than a dense superposition of all updates.

2.7.3 LoRA Adapter Merging

Merging LoRA adapters introduces additional considerations beyond those studied for full fine-tuned models. Tang et al. [59] showed that LoRA adapters are less disentangled than full fine-tuned model deltas when the adapters have been trained on heterogeneous tasks, and that linear combination of such adapters can produce interference that degrades performance below that of any individual adapter. This interference effect grows with the number of merged adapters and with the task diversity of the training data.

Chronopoulou et al. [9] identified shared initialization as an important mitigating factor: adapters trained from the same pretrained base on related tasks exhibit substantially lower interference under linear merging than adapters trained on disparate tasks, because their updates tend to lie in more compatible subspaces of the parameter space.

Arithmetic operations on LoRA adapters have been studied as a broader framework for adapter composition. LoraHub [25] demonstrated that a weighted combination of LoRA adapters trained on different tasks can be assembled using a small amount of task-specific validation data to determine the combination weights, enabling few-shot task specialization without any gradient-based fine-tuning. This result showed that the weight space of LoRA adapters supports meaningful arithmetic composition when the constituent adapters are drawn from a compatible distribution of related tasks.

Chapter 3: Quality-Stratified Low-Rank Adaptation

This chapter describes the Quality-Stratified Low-Rank Adaptation (QS-LoRA) method and the end-to-end pipeline in which it is embedded. The pipeline takes a document collection as input and produces a single dense model checkpoint as output, without requiring human annotation or access to external API providers at inference time. Section 3.1 provides a high-level overview of the full pipeline. Sections 3.2–3.6 describe each stage in detail. Section 3.7 describes the distilled local judge that enables air-gapped execution of the quality scoring step.

3.1 Pipeline Overview

The QS-LoRA pipeline consists of six sequential stages. Figure 3.1 shows the complete flow.

1. **Synthetic QA generation.** A fine-tuned small generator model produces question-answer pairs from document chunks or passages.
2. **Quality scoring.** Each generated pair is scored on four dimensions by an LLM judge. Two judge options are available: Claude Haiku via AWS Bedrock for standard use, or a locally distilled Llama-3.2-3B adapter for air-gapped deployment.

3. **Tier assignment.** Pairs are assigned to one of four categories, high, medium, low, or drop, based on their quality scores.
4. **Tier-specific adapter training.** Three separate LoRA adapters are trained on the high, medium, and low tier subsets at ranks $r = 32$, $r = 16$, and $r = 8$ respectively. Dropped pairs are excluded from training entirely.
5. **Tier-weighted dense merge.** The three adapters are combined into a single dense model checkpoint using fixed tier-importance weights.
6. **Deployment.** The merged checkpoint is deployed for inference. No LoRA adapter is required at inference time.

The order is deliberate: quality scoring must follow generation because scores depend on the generated content, tier assignment must precede training because it determines which adapter each pair trains, and the dense merge must follow all three adapter training runs because it combines their learned updates into a single deployable artifact.

3.2 Synthetic Question-Answer Generation

The first stage of the pipeline produces synthetic question-answer pairs from document content. Since private document collections have no associated human-annotated question-answer datasets, synthetic generation is the only scalable path to building fine-tuning supervision.

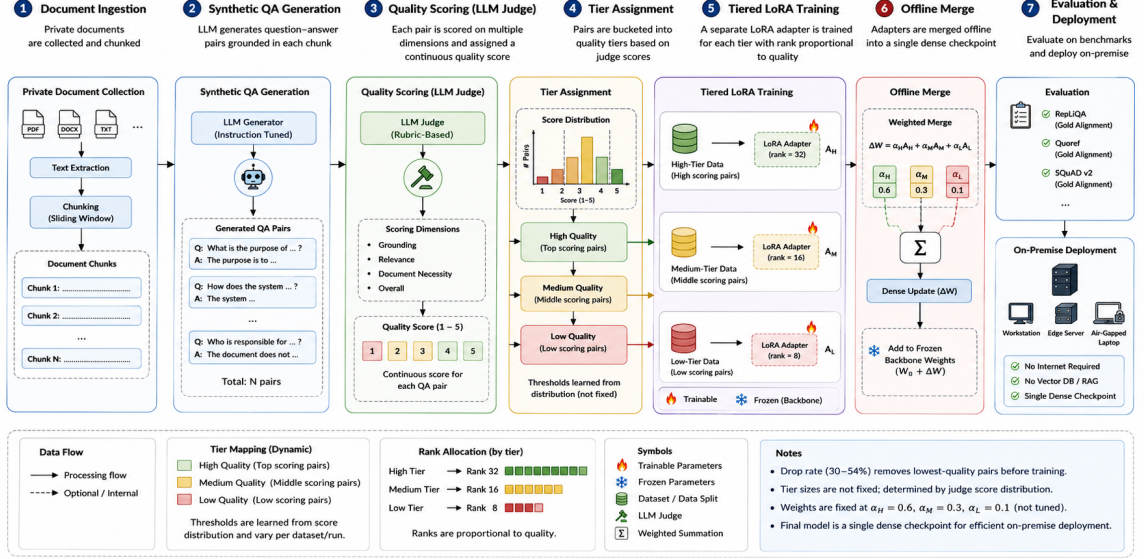


Figure 3.1: The QS-LoRA end-to-end pipeline. Starting from a private document collection, the pipeline proceeds through seven stages: document ingestion and chunking, synthetic QA generation, quality scoring by an LLM judge on four rubric dimensions (Grounding, Relevance, Document Necessity, Overall), tier assignment into high/medium/low quality bins based on judge score distribution, tier-specific LoRA adapter training at ranks proportional to quality ($r = 32, 16, 8$), offline tier-weighted dense merge ($\Delta W = 0.6A_H + 0.3A_M + 0.1A_L$), and on-premise deployment as a single dense checkpoint requiring no internet access, no retrieval index, and no adapter overhead at inference time.

3.2.1 Generator Model

The generator is a Llama-3.2-3B-Instruct model fine-tuned on a curated seed corpus of 9,462 document-grounded question-answer triples drawn from four scientific QA datasets: QASPER [12] (5,049 triples), SciDQA [54] (2,937), PubMedQA [26] (1,000), and PeerQA [16] (476). The fine-tuning task is excerpt-to-JSON generation: given a document passage, produce a JSON object with keys **question** and **answer**. These datasets are used solely for generator fine-tuning and do not appear anywhere in the evaluation pipeline; the evaluation benchmarks (RepLiQA, Quoref, SQuAD v2)

are entirely separate, precluding any data leakage between generator training and downstream evaluation. A fine-tuned small generator is used rather than the judge model (Claude Haiku) or a larger API-based model for three reasons. First, it keeps the generation step fully on-premise – no document content is transmitted to external providers. Second, a small fine-tuned model is substantially cheaper at scale than per-token API calls for tens of thousands of generation requests. Third, the generator and judge are deliberately kept in separate model families (Llama vs Claude) to avoid a self-preference loop where the same model that generates answers also scores them. Using the judge model as the generator would collapse this separation.

After fine-tuning, the LoRA adapter is merged into the base model to produce a dense checkpoint (**merged-normalized-generator**), which is served via a local vLLM endpoint for generation. Using a small 3B generator rather than a large API model keeps the generation step on-premise, avoids per-token API cost, and ensures that document content is not transmitted to external providers.

The generator was trained on scientific document QA datasets (QASPER, SciDQA, PubMedQA, PeerQA) but is deployed across document types spanning enterprise knowledge bases (RepLiQA), Wikipedia co-reference passages (Quoref), and extractive QA corpora (SQuAD v2). This domain mismatch between training and deployment is intentional: the generation task, reading a passage and producing a grounded question-answer pair, is structurally domain-agnostic. The fine-tuning teaches the generator format discipline and grounding constraints, not domain-specific content. The quality scoring step further mitigates potential domain bias by independently verifying groundedness, relevance, and document necessity for every pair regardless of source domain.

The generator choice does affect the type of questions produced. A fine-tuned small model trained on scientific QA datasets (QASPER, SciDQA, PubMedQA, PeerQA) tends to produce chunk-level lookup questions – questions whose answers are localized to a specific passage. It rarely produces multi-hop reasoning questions that require synthesizing information across multiple document sections. This limits the diversity of the generated question distribution and is acknowledged as a limitation in Section 6.3. The comparison with a Haiku API generator in Section 5.6.4 confirms that a higher-quality generator produces better coverage (97.8% vs 84.5% lexical) and higher eval GA (4.03 vs 3.75), motivating future work on improved generation pipelines.

3.2.2 Generation Prompt

The generator is prompted with a system message and a user message. The system message instructs the model to write exam-style question-answer pairs for scientific text and constrains answers to facts contained in the provided excerpt only. The user message presents the document context and requests exactly one question-answer pair in JSON format without markdown fences. Temperature is set to 0.7 to introduce controlled diversity across generated pairs while maintaining coherence with the source passage. One pair is generated per document context. At evaluation time, fine-tuned models generate answers using greedy decoding (temperature = 0), ensuring deterministic, reproducible outputs and eliminating sampling stochasticity as a confound in evaluation results.

3.2.3 Document Preparation

Document preparation varies by corpus. For benchmark datasets (RepLiQA, Quoref, SQuAD v2), passages are used as-is from the benchmark JSONL files, with

one generation call per unique passage. No additional chunking is applied to these corpora since each passage is already a self-contained context unit.

For the OhioLine agricultural extension corpus, raw HTML fact sheets are converted to plain text and segmented using a sliding window chunker with a maximum chunk size of 6,000 characters and an overlap of 200 characters. Markdown tables within passages are treated as atomic units and are never split mid-row. This produces 1,552 chunks from 701 source passages (an average of 2.2 chunks per passage). One question-answer pair is generated per chunk.

3.3 Quality Scoring

Each generated question-answer pair is scored by an LLM judge before tier assignment. The standard judge is Claude Haiku 4.5 accessed via AWS Bedrock (`us.anthropic.claude-haiku-4-5-20251001-v1:0`). Section 3.7 describes a locally distilled alternative for air-gapped deployments.

Judge selection rationale. Claude Haiku 4.5 was chosen as the quality judge for three reasons. First, it is cost-efficient at scale: the entire thesis scoring corpus of 74,115 pairs cost \$84 in total. Second, and most importantly, it belongs to a different model family (Anthropic Claude) than the generator (Meta Llama-3.2-3B), which avoids a direct self-preference circularity loop. If the same model family generated and scored the pairs, high scores might reflect the generator’s own distributional preferences rather than objective quality. Third, Haiku’s instruction-following reliability produces consistent structured JSON output across long scoring runs without format failures.

A separate but related concern is that the same judge is used both for quality scoring during training data preparation and for Gold Alignment evaluation of model

outputs. This dual role is discussed explicitly in Section 4.3.1 and acknowledged as a limitation in Section 6.3. The key mitigation is that the backbone models being evaluated (Llama, Qwen2.5, Gemma-3) are all from different model families than the judge, so there is no direct self-preference loop between the models being evaluated and the judge scoring them.

Four-dimension rubric rationale. The four scoring dimensions were selected to cover the distinct failure modes observed in synthetic document-grounded QA generation: grounding failures (answer contradicts the source), relevance failures (answer does not address the question), document-necessity failures (question is answerable without the document), and overall quality failures (pair is structurally correct but low-value as a training signal). Each dimension targets a failure mode the others do not capture, making the set jointly sufficient for tier assignment without redundancy.

3.3.1 Scoring Rubric

The judge evaluates each pair in a single API call and returns a JSON object containing integer scores on four dimensions, each rated on a 1–5 scale with explicit anchors at 1, 3, and 5:

1. **Grounding.** Is the answer directly supported by the provided context? A score of 5 indicates every claim is present in the context; a score of 1 indicates the answer contradicts or ignores the context entirely.
2. **Relevance.** Does the answer address the question? A score of 5 indicates the answer is directly responsive; a score of 1 indicates the answer is off-topic or evasive.

3. **Document necessity.** Is the specific source document required to answer the question, or could the question be answered from general knowledge? A score of 5 indicates the question is impossible to answer without this document; a score of 1 indicates the question tests only general world knowledge that any model would know.
4. **Overall.** Is this a valuable training example? A score of 5 indicates a high-quality training pair; a score of 1 indicates a poor or misleading pair. This dimension penalizes generic knowledge question-answer pairs regardless of their scores on other dimensions.

The judge prompt instructs the model to score conservatively: a score of 5 should be reserved for essentially perfect pairs, 3 for pairs that are partially useful but clearly imperfect, and 1 for pairs that are clearly wrong, off-topic, or unsupported. The judge also returns a boolean `refuse_expected` flag and a `brief_reason` string. All four dimensions are scored in a single API call to minimize cost and latency.

3.3.2 Score Distribution

Table 3.1 reports the tier distribution across all three evaluation datasets after quality scoring. The drop rate ranges from 30% on SQuAD v2 to 54% on Quoref, confirming that a non-trivial fraction of synthetically generated pairs are of insufficient quality for training and motivating the stratification approach.

3.4 Tier Assignment

The tier assignment rules were designed to reflect a principled quality ordering rather than an empirically tuned threshold. The grounding dimension is treated

Table 3.1: Quality tier distribution after Haiku scoring across the three evaluation datasets. Drop rate = pairs excluded from training entirely. SFT rows = pairs used for adapter training after 90/10 document-level train/val split.

Dataset	Judged	High	Medium	Low	Drop	Drop %	SFT rows (B3)
RepLiQA	13,770	7,668	1,274	643	4,139	30%	13,770
Quoref	7,118	2,532	372	364	3,843	54%	7,118
SQuAD v2	39,787	20,410	1,870	2,702	14,765	37%	39,787

as a hard gate because an answer that contradicts the source document is actively harmful as a training signal, regardless of its scores on other dimensions. The high tier requires all four dimensions to score at the satisfactory level (≥ 4) so that it contains only pairs where every quality axis is met. The thresholds (`high_min` = 4, `low_max` = 2) were set before any evaluation runs and were not tuned on held-out sets. The fact that drop rates vary from 30% (RepLiQA) to 54% (Quoref) across datasets reflects genuine variation in synthetic generation difficulty across document types, not threshold overfitting.

A full sensitivity analysis of the thresholds is beyond the current scope, but a qualitative argument for robustness can be made: shifting `high_min` from 4 to 3 would expand the high tier substantially (many medium-quality pairs score at least 3 on all dimensions), likely reducing the quality signal that distinguishes the high-tier adapter. Shifting `low_max` from 2 to 3 would move many medium pairs to the low tier, concentrating more data in the low-rank adapter and potentially reducing fine-tuning quality. The chosen thresholds represent a natural break in the 1–5 scoring scale: below the midpoint (low), above the satisfactory level (high), and everything else (medium). The consistency of QS-LoRA’s advantage across three datasets with very

different drop rates (30–54%) provides indirect evidence that the tier assignment is capturing a real quality signal rather than an artifact of the specific threshold values.

After scoring, each pair is assigned to a tier according to the following deterministic rules, applied in priority order:

1. **Drop.** The pair is excluded if the answer field is empty, null, or not-a-number, or if the grounding score is ≤ 2 . A grounding score of 2 or below indicates the answer contradicts or substantially departs from the source document, making the pair not only uninformative but potentially harmful as a training signal.
2. **Low.** The pair is assigned to the low tier if any of the four dimensions scores ≤ 2 . Since the drop rule already handles grounding ≤ 2 , low-tier pairs are those with a weak score on relevance, document necessity, or overall quality, while remaining minimally grounded.
3. **High.** The pair is assigned to the high tier if all four dimension scores are ≥ 4 . This requires the pair to be well-grounded, directly relevant, document-specific, and of high overall training value.
4. **Medium.** All remaining pairs, those not dropped, not low, and not high, are assigned to the medium tier. Medium pairs are usable but imperfect: they may have one dimension scoring 3, or a mix of 3s and 4s without achieving all ≥ 4 .

Table 3.2 reports the number of training pairs per tier after the document-level 90/10 train/validation split. Tier assignment is performed before splitting, and the split is stratified at the document level to prevent data leakage between training and validation sets.

Table 3.2: SFT training rows per tier after 90/10 document-level train/val split. B3 trains on all pairs. QS-LoRA trains three separate adapters, one per tier.

Dataset	B3 (all)	QS High	QS Medium	QS Low
RepLiQA	12,393	6,901	1,147	579
Quoref	6,406	2,288	336	326
SQuAD v2	35,808	18,348	1,681	2,437

3.5 Tier-Specific LoRA Adapter Training

Three independent LoRA adapters are trained, one per quality tier. The central design principle is that adapter rank should be proportional to the quality of the training signal: high-quality pairs justify a higher-capacity weight update, while low-quality pairs should contribute only a low-capacity correction.

3.5.1 Rank and Scaling

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, each adapter parameterizes the weight update as:

$$\Delta W = \frac{\alpha}{r} B A, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k} \quad (3.1)$$

where r is the adapter rank and α is the LoRA scaling factor. The scaling ratio α/r controls the effective magnitude of the weight update relative to rank. In QS-LoRA, $\alpha = 2r$ for all three tiers, so the scaling ratio is fixed at 2 across tiers. The effective capacity of each adapter therefore scales with r :

The rank values $r \in \{32, 16, 8\}$ were chosen as integer multiples of a base rank of $r = 8$, following the convention in prior LoRA work that ranks be small powers of two or multiples thereof [24]. The 4:2:1 ratio across tiers was selected heuristically to give

Table 3.3: Rank and scaling configuration per quality tier.

Tier	Rank r	Scaling α	Trainable params (per matrix)
High	32	64	$32(d + k)$
Medium	16	32	$16(d + k)$
Low	8	16	$8(d + k)$

the high-quality tier substantially more capacity than the low tier while keeping the medium tier at the standard LoRA default ($r = 16$). Pilot experiments comparing 32/16/8 against a uniform $r = 16$ baseline confirmed that tier differentiation improves performance on Quoref (+0.08 GA) while equal ranks at $r = 16$ achieve similar results on RepLiQA (see equal-rank ablation in Section 5.6.5). Alternative rank ratios such as $r = 24/12/6$ or $r = 16/8/4$ were not systematically evaluated; whether these would further improve or degrade performance is left as future work. The ranks $r \in \{32, 16, 8\}$ correspond to a roughly $2\times$ step-down in capacity between adjacent tiers, giving high-tier pairs a clearly larger adaptation subspace while keeping the total parameter budget across all three adapters roughly comparable to a single uniform LoRA adapter at $r = 16$.

3.5.2 Target Modules

LoRA adapters are applied to all seven linear projection modules within each transformer block: the query, key, value, and output projections of the self-attention sublayer (`q_proj`, `k_proj`, `v_proj`, `o_proj`), and the gate, up, and down projections of the feed-forward sublayer (`gate_proj`, `up_proj`, `down_proj`). Applying adapters to all projection modules, rather than only the query and value projections as in the original LoRA paper, has been shown to improve adaptation quality on generative

tasks [24]. These seven modules collectively cover both the attention mechanism and the feed-forward transformation at every layer, the subspaces where document-grounded factual recall and reasoning behavior have been shown to be most sensitive to.

3.5.3 Training Configuration

All three adapters share the same training configuration except for rank and scaling.

Table 3.4 summarizes the full hyperparameter set.

Table 3.4: Training hyperparameters shared across all three tier-specific adapters. Rank and scaling (α) differ per tier as shown in Table 3.3.

Hyperparameter	Value
Optimizer	AdamW (torch default)
Learning rate	2×10^{-4}
Effective batch size	8 (1 per device $\times$ gradient accumulation 8)
Epochs	3
Maximum sequence length	4,096 tokens
Precision	bfloat16
LoRA dropout	0.05
Warmup steps	0
Weight decay	0.0
Train/val split	90/10 document-level, seed 42
Random seed	42

Training is performed on a single NVIDIA A100-SXM4-80GB GPU (Ohio Super-computer Center ASCEND cluster, partition **quad**) for all models in the 1B–14B range. The Llama-3.1-70B model is trained with 4-bit QLoRA on four A100-80GB GPUs using 256 GB of node memory. Each condition is trained once (single seed); bootstrap

confidence intervals are computed over evaluation items rather than independent training runs.

3.6 Tier-Weighted Dense Merge

After training, the three tier-specific adapters are combined into a single dense model checkpoint. This step serves two purposes: it produces a single deployable artifact from three separately trained adapters, and it eliminates the inference overhead of serving a LoRA adapter at runtime.

3.6.1 Merge Formula

Let ΔW_{high} , ΔW_{med} , and ΔW_{low} denote the weight update matrices of the high, medium, and low tier adapters respectively. The merged weight update is:

$$\Delta W_{\text{merged}} = 0.6 \cdot \Delta W_{\text{high}} + 0.3 \cdot \Delta W_{\text{med}} + 0.1 \cdot \Delta W_{\text{low}} \quad (3.2)$$

The merged update is added to the frozen pretrained weights W_0 to produce the final deployed weight matrix $W = W_0 + \Delta W_{\text{merged}}$.

The merge coefficients (0.6, 0.3, 0.1) are set to reflect the relative importance of each quality tier: the high-quality adapter receives the largest weight, the medium adapter a moderate weight, and the low adapter a small but non-zero weight. The coefficients sum to 1.0, preserving the overall scale of the merged update relative to the individual adapter updates. These coefficients are fixed across all backbone models and datasets and are not tuned per-run. The values 0.6/0.3/0.1 were chosen as a round, interpretable default reflecting a 6:3:1 quality weighting ratio; the merge weight ablation (Section 5.6.1) confirms that any non-equal scheme respecting the

quality ordering performs within 0.15 GA of this default, so the exact values are not critical as long as the ordering is preserved.

To assess sensitivity to this choice, five alternative weighting schemes were evaluated across all three reference datasets on Llama-3.2-3B, and additionally across eight backbone models (1B–14B) on all three datasets. The five schemes are: equal weights (1.0/1.0/1.0), frequency-proportional weights (count-weighted by tier size), high+medium only (0.67/0.33/0.0), low-heavy weights (0.4/0.4/0.2), and inverted weights (0.1/0.3/0.6).

The sweep yields four findings. First, among all non-equal alternatives, GA scores vary by less than 0.05 points on RepLiQA and Quoref, and tier weighting (0.6/0.3/0.1) is best or tied-best on all 24 complete cross-model cells. The qualitative principle, giving the most weight to the highest-quality adapter and the least to the lowest-quality adapter, is what matters, not the precise coefficient values. Second, equal weighting (1.0/1.0/1.0) is the single exception: it produces catastrophic degradation (GA 2.39 on RepLiQA, 2.21 on Quoref, 1.78 on SQuAD v2) due to conflicting update directions from adapters trained on qualitatively different data subsets. This failure mode generalizes across all nine evaluated backbone models without exception. Third, the inverted scheme (0.1/0.3/0.6) never wins and is consistently among the worst non-equal alternatives, confirming that quality-ordering of weights is necessary. Fourth, the merge weights are fixed across all backbone models and datasets without per-deployment tuning; the cross-model sweep shows this fixed default performs at least as well as any alternative on every evaluated model-dataset combination.

3.6.2 Inference Efficiency

Because the merged adapter weights are folded directly into the base model parameters before deployment, the deployed model is structurally identical to the unmodified base model. During inference, every transformer layer performs a single matrix multiplication per projection: Wx where $W = W_0 + \Delta W_{\text{merged}}$. In contrast, serving a LoRA adapter dynamically requires computing $W_0x + \frac{\alpha}{r}BAx$ at every layer: two matrix multiplications plus an addition per projection, per forward pass. Across seven adapted modules per transformer block and 28–80 blocks depending on model size, this overhead accumulates with every generated token. The dense merge eliminates this overhead entirely by absorbing ΔW_{merged} into W_0 at deployment time.

Throughput improvement was measured on a single NVIDIA A100-SXM4-80GB GPU at batch size 1, context length 2,048 tokens, comparing merged-dense inference against PEFT dynamic adapter serving under equivalent settings. The measured improvement is 2.3–3.2 $\times$ depending on model size. This comparison is hardware-specific; the relative benefit decreases at larger batch sizes where compute dominates over per-token overhead.

3.7 Distilled Local Judge

The standard quality scoring step requires access to Claude Haiku via AWS Bedrock, which introduces a per-call API cost and transmits document content to an external provider. For organizations that require fully air-gapped operation, where no data may leave the local infrastructure, this dependency is unacceptable.

To address this, a local judge model is trained to replicate the Haiku scoring step using only on-premise compute. The distilled judge is a Llama-3.2-3B-Instruct model

fine-tuned with LoRA on pairs of document contexts with their corresponding Haiku-assigned quality scores. The training corpus consists of 56,794 scored question-answer pairs drawn from RepLiQA, Quoref, SQuAD v2, and OhioLine, covering all four evaluation domains used in this thesis.

3.7.1 Training

The distilled judge is trained to predict the same four-dimensional rubric scores (grounding, relevance, document_necessity, overall) that the Haiku judge produces, given the same input (document context, question, answer). The training task is identical in format to the Haiku scoring call: the judge receives the triple and returns a JSON object with integer scores on each dimension. Haiku scores serve as the supervision signal; no human annotation is used.

The judge was trained in four successive versions (v1–v4), each expanding the training corpus by adding a new domain. Version v4 is the production judge, trained on the full 56,794-pair corpus spanning all four domains.

3.7.2 Evaluation

The distilled judge is evaluated on an out-of-domain holdout of 511 OhioLine question-answer pairs that were not seen during any training version. The evaluation measures tier assignment agreement between the distilled judge and Haiku on the same inputs. Table 3.5 reports the results.

A Cohen’s κ of 0.62 indicates substantial agreement [32] between the distilled judge and Haiku, and a Spearman ρ of 0.79 confirms strong rank-order correlation in continuous score predictions. These results demonstrate that the quality scoring step can be internalized into a small local model without significant degradation in tier

Table 3.5: Distilled local judge (v4) evaluation against Haiku on the OhioLine out-of-domain holdout ($n = 511$).

Metric	Value
Tier agreement	78.5%
Cohen’s κ	0.62
Spearman ρ	0.79
Training pairs	56,794
Holdout size	511

assignment accuracy relative to the API judge. These statistics are sufficient to trust the judge for tier assignment, but they do not imply full equivalence to Haiku on all evaluation tasks

The distilled judge is a drop-in replacement for the Haiku scoring step in the QS-LoRA pipeline. All other pipeline stages remain identical regardless of which judge is used.

Chapter 4: Experimental Setup

This chapter describes the experimental configuration used to evaluate Quality-Stratified Low-Rank Adaptation. Section 4.1 describes the three evaluation datasets and how each is used for synthetic training and human-annotated evaluation. Section 4.2 defines the three experimental conditions compared in this thesis. Section 4.3 describes the Gold Alignment evaluation metric and the bootstrap confidence interval methodology. Section 4.4 lists the ten backbone models evaluated in the generalization study. Section 4.5 describes the hardware and infrastructure used for all experiments.

4.1 Evaluation Datasets

Three document-grounded question-answering benchmarks are used for evaluation, each testing a different aspect of the model’s ability to answer questions from a private document collection. Table 4.1 summarizes the key statistics.

4.1.1 RepLiQA

RepLiQA [47] is a benchmark of human-authored question-answer pairs grounded in documents drawn from a proprietary enterprise knowledge base. It closely approximates the private document question-answering setting that motivates this thesis: documents are not publicly available on the internet, questions require consulting the source

Table 4.1: Summary of the three evaluation datasets. Generated pairs = total synthetic QA pairs produced before quality filtering. B3/B5 trains on all generated pairs (no filtering, after 90/10 train/val split). QS-LoRA trains on non-drop pairs only (high + medium + low tiers, after 90/10 split). Eval rows = human-authored QA pairs used for GA evaluation.

Dataset	Generated pairs	Drop rate	B3/B5 SFT rows	QS-LoRA SFT rows	Eval rows
RepLiQA	13,770	30%	12,393	8,627	2,000
Quoref	7,118	54%	6,406	2,950	2,418
SQuAD v2	39,787	37%	35,808	22,466	11,873

document, and answers are written by human annotators with access to the full document.

The benchmark is distributed as five document splits (RepLiQA-0 through RepLiQA-4). Splits 0–3 are used as the in-distribution corpus; split 4 is reserved as an out-of-distribution generalization set and is excluded from all training and main evaluation runs reported in this thesis.

Synthetic training. All 14,364 unique documents from splits 0–3 are used as the synthetic generation corpus. One synthetic question-answer pair is generated per document using the fine-tuned Llama-3.2-3B generator, producing 13,770 pairs after Haiku quality scoring. Models are fine-tuned on these synthetic pairs; no human-authored RepLiQA answers are used as SFT targets.

Evaluation. Models are evaluated on a fixed subset of 2,000 human-authored question-answer pairs sampled from splits 0–3 (100 documents per split, approximately five questions per document, seed 42). This subset is held constant across all models and conditions. Document overlap between the synthetic training pool and the evaluation document set is possible by design; what is held out at evaluation time is the human-authored questions and answers, not the documents themselves.

4.1.2 Quoref

Quoref [11] is a reading comprehension dataset that requires resolving co-references across multiple sentences to answer questions correctly. A question may ask about an entity that is referred to by different names or pronouns across the passage, requiring the model to track identity chains rather than perform simple span extraction. This makes Quoref substantially harder than factoid extraction benchmarks and tests whether fine-tuning on synthetic data transfers to multi-hop reasoning tasks.

Synthetic training. The train and validation passage sets are combined into a single knowledge base. Two synthetic question-answer pairs are generated per unique passage, producing 7,118 Haiku-scored pairs. LoRA adapters are trained on these synthetic pairs; human Quoref annotations are not used as SFT targets.

Evaluation. Models are evaluated on the full Quoref validation set (2,418 human question-answer pairs). No public labeled test set is available for Quoref; the validation set is the standard evaluation target in the literature.

4.1.3 SQuAD v2

SQuAD v2 [52] extends the original Stanford Question Answering Dataset with a set of unanswerable questions: questions that are plausibly related to the passage but cannot be answered from its content. A model must not only extract correct answers when they exist but also recognize and refuse to answer when the passage does not support a response. This dual requirement makes SQuAD v2 the hardest of the three evaluation datasets and tests the model’s calibration on the boundary between answerable and unanswerable questions.

Synthetic training. The train and validation passage sets are combined into a knowledge base. Two synthetic pairs are generated per unique passage, producing 39,787 Haiku-scored pairs—the largest synthetic training pool of the three datasets. LoRA adapters are trained on these synthetic pairs; human SQuAD v2 annotations are not used as SFT targets.

Evaluation. Models are evaluated on the full SQuAD v2 validation set (11,873 human question-answer pairs). The SQuAD v2 test set labels are not publicly available; the validation set is the standard evaluation target.

4.2 Baseline Conditions

Three experimental conditions are compared across all datasets and backbone models. Table 4.2 summarizes the key differences.

Table 4.2: The three experimental conditions compared in this thesis. B3 and B5 train on all generated pairs including drop-tier pairs. QS-LoRA trains on non-drop pairs only, stratified by quality tier, and additionally requires a one-time Haiku quality scoring step per dataset.

Condition	Method	Rank	Data
B3	Uniform LoRA	$r = 16, \alpha = 32$ (fixed)	All pairs
B5	AdaLoRA	init $r = 16$, adaptive pruning	All pairs
Ours	QS-LoRA (this thesis)	$r \in \{32, 16, 8\}$ per tier	Tier-stratified

To be explicit: B3 and B5 train on all generated synthetic pairs including those that would be assigned to the drop tier under QS-LoRA’s quality scoring – that is, pairs with grounding score ≤ 2 or empty answers are included in B3/B5 training but

excluded from QS-LoRA training. This means B3 and B5 train on more data than QS-LoRA (see Table 4.1 for per-dataset counts), making them conservative baselines.

4.2.1 B3: Uniform LoRA

B3 is the standard LoRA fine-tuning baseline. A single adapter is trained at fixed rank $r = 16$ with scaling $\alpha = 32$ on all the pairs. All pairs receive equal weight during training regardless of their Haiku quality scores. B3 does not require the Haiku quality scoring step and can be trained directly from the raw synthetic QA pool. B3 represents the current standard practice for fine-tuning language models on synthetic document-grounded data.

4.2.2 B5: AdaLoRA

B5 is the AdaLoRA baseline [71]. AdaLoRA parameterizes each weight update as a singular value decomposition $\Delta W = P\Lambda Q$ and dynamically prunes singular values based on their importance scores during training. The rank budget is initialized at $r = 16$ and redistributed across weight matrices as training progresses, reducing rank in less sensitive layers and preserving it in more sensitive ones. AdaLoRA is configured with $t_{\text{init}} = 10\%$ of total steps, $t_{\text{final}} = 90\%$, and $\Delta t = 10$ steps between pruning decisions. Like B3, AdaLoRA trains on all pairs without quality stratification.

B5 is included as a strong adaptive rank allocation baseline to establish whether the benefit of QS-LoRA arises from rank differentiation in general or specifically from the quality-stratified assignment of rank to data subsets.

4.2.3 Ours: QS-LoRA

The proposed method, described in full in Chapter 3. Three separate LoRA adapters are trained at ranks $r = 32$, $r = 16$, and $r = 8$ on the high, medium, and low quality tier subsets respectively. After training, the three adapters are merged into a single dense checkpoint using tier-weighted coefficients (0.6, 0.3, 0.1). QS-LoRA requires the Haiku quality scoring step as a one-time preprocessing cost per dataset (\$13.33 for RepLiQA, \$9.07 for Quoref, \$40.11 for SQuAD v2 at on-demand Bedrock pricing), which is amortized across all backbone models trained on that dataset.

4.3 Evaluation Metric

4.3.1 Gold Alignment

All models are evaluated using Gold Alignment (GA), a reference-based metric that measures the semantic alignment between a model’s generated answer and a human-authored gold answer. GA is scored by a separate LLM judge (Claude Haiku 4.5 via AWS Bedrock, prompt version `qa_judge_rubric/v3_eval_gold`) on a 1–5 integer scale. The judge evaluates five dimensions for each model response:

1. **Grounding.** Is every claim in the response directly supported by the provided context? (1 = contradicts or ignores context; 5 = every claim present in context)
2. **Relevance.** Does the response address the question with the same intent as the gold answer? (1 = off-topic or evasive; 5 = directly answers with same intent as gold)

3. **Gold alignment.** Does the response convey the same meaning as the gold answer, preserving all critical facts? (1 = wrong or invents facts when gold refuses; 5 = equivalent meaning, all critical facts preserved)
4. **Document necessity.** Is the source document required to produce this response? (1 = pure general knowledge; 5 = impossible without this document)
5. **Overall.** Is this response an excellent and reliable answer? (1 = poor or misleading; 5 = excellent and reliable)

The primary reported metric is the mean `gold_alignment` score across all evaluation questions, referred to throughout this thesis as the GA score. The judge prompt instructs conservative scoring: a score of 5 is reserved for essentially perfect responses, 3 for responses that are partially useful but clearly imperfect, and 1 for responses that are clearly wrong, off-topic, or unsupported. The standard `v3_eval_gold` prompt version is used for all main result tables; the debiased variant (which swaps gold and prediction block order and averages) is not used in the reported results.

4.3.2 Why Gold Alignment Over Lexical Metrics

Standard automatic metrics such as ROUGE [39] and exact match measure lexical overlap between a generated response and a reference string. These metrics are poorly suited to open-ended generative question answering for two reasons. First, a correct answer to a document-grounded question can be expressed in many different ways; a response that paraphrases the gold answer correctly will score near zero on exact match and poorly on ROUGE even if its meaning is equivalent. Second, exact match heavily penalizes refusal behavior on unanswerable questions (SQuAD v2): a model

that correctly declines to answer an unanswerable question using different phrasing than the gold “unanswerable” label receives no credit.

GA addresses both limitations by evaluating semantic equivalence rather than surface overlap and by explicitly scoring whether the model correctly refuses when the gold answer refuses. This makes GA more appropriate for the private document fine-tuning evaluation setting where both response quality and calibrated refusal are important.

4.3.3 Bootstrap Confidence Intervals

Statistical significance of GA differences between conditions is assessed using paired bootstrap confidence intervals [17]. For each pair of conditions (e.g. Ours vs. B3), 10,000 bootstrap resamples are drawn over the set of evaluation question IDs with replacement, and the mean GA difference is recomputed on each resample. The 95% confidence interval is the 2.5th to 97.5th percentile of the resulting distribution. A difference is considered statistically significant if the 95% CI excludes zero.

Each experimental condition is trained once with a fixed random seed (seed=42) due to compute budget constraints. Item-level bootstrap confidence intervals (10,000 resamples) quantify uncertainty over evaluation items given fixed model weights, but they do not capture training variance across seeds. Results reflect carefully controlled single-seed experiments; whether gains hold across different random seeds is an open question and multi-seed replication is left as future work.

4.4 Backbone Models

Ten open-weight backbone models are evaluated in the generalization study, spanning three model families at four parameter scales. Table 4.3 lists all models with their key properties.

Table 4.3: Backbone models evaluated in the generalization study. QLoRA = 4-bit quantized LoRA used due to VRAM constraints. RepLiQA and Quoref results are available for all ten models; SQuAD v2 results are available for the 1B–14B models only.

Model	Family	Params	QLoRA	GPUs	RepLiQA	Quoref
Llama-3.2-1B	Llama	1B	No	1	✓	✓
Llama-3.2-3B	Llama	3B	No	1	✓	✓
Llama-3.1-8B	Llama	8B	No	1	✓	✓
Llama-3.1-70B	Llama	70B	Yes	4	✓	✓
Qwen2.5-3B	Qwen	3B	No	1	✓	✓
Qwen2.5-7B	Qwen	7B	No	1	✓	✓
Qwen2.5-14B	Qwen	14B	No	1	✓	✓
Gemma-3-1B	Gemma	1B	No	1	✓	✓
Gemma-3-4B	Gemma	4B	No	1	✓	✓
Gemma-3-12B	Gemma	12B	No	1	✓	✓

The Llama-3.2-3B model serves as the primary reference backbone throughout this thesis. The equal-rank ablation and high-only ablation are reported on this model only. The merge weight sweep is reported on all nine backbone models across all three datasets. All other GA results are reported in the cross-model generalization study in Chapter 5.

The Llama-3.1-70B model is trained using 4-bit QLoRA on four A100-80GB GPUs. SQuAD v2 results for the 70B model are not available as the run was not completed within the thesis timeline; RepLiQA and Quoref results are reported.

4.5 Hardware and Infrastructure

All training, inference, and evaluation runs are conducted on the Ohio Supercomputer Center (OSC) ASCEND cluster using NVIDIA A100-SXM4-80GB GPUs (80 GB VRAM per GPU), partition `quad`, account PAS2699. Models in the 1B–14B parameter range are trained and evaluated on a single A100-80GB GPU. The Llama-3.1-70B model is trained with 4-bit QLoRA on four A100-80GB GPUs with 256 GB of node memory.

All experiments use PyTorch with the HuggingFace Transformers and PEFT libraries. LoRA and QLoRA adapters are trained using TRL `SFTConfig`. AdaLoRA adapters are trained using the PEFT AdaLoRA implementation. Model inference for evaluation is performed in bfloat16 precision on a single A100-80GB GPU for all backbone sizes except 70B, which uses 4-bit quantization at inference time. At evaluation time, all fine-tuned models generate answers using greedy decoding (temperature = 0, no nucleus sampling), which produces deterministic outputs for a given input and ensures reproducibility. Greedy decoding was chosen over sampling to eliminate stochasticity as a confound in the evaluation results; a different random seed for sampling could otherwise produce different outputs and different GA scores on the same model checkpoint.

Quality scoring calls to Claude Haiku 4.5 are made via the AWS Bedrock API (`us.anthropic.claude-haiku-4-5-20251001-v1:0`). RepLiQA scoring used Anthropic batch processing (50% discount); Quoref, SQuAD v2, and OhioLine scoring used Bedrock on-demand pricing. Total Haiku scoring cost across all datasets used in this thesis was \$84.40.

Chapter 5: Results and Analysis

This chapter presents the experimental results across all evaluation datasets, backbone models, and analysis dimensions. Section 5.1 reports Gold Alignment results for the seven small backbone models (1B–8B parameters) across all three datasets. Section 5.2 reports results for the three large backbone models (12B–70B). Section 5.3 quantifies the gap between open fine-tuned models and closed-model ceiling references. Section 5.4 presents the end-to-end training cost analysis including quality scoring overhead. Section 5.5 provides a mechanistic explanation for the scale-dependent behavior of QS-LoRA through adapter weight geometry analysis. Section 5.6 presents ablation studies on merge strategy and tier data contribution.

5.1 Results: Small Models (1B–8B)

Headline. QS-LoRA improves Gold Alignment over uniform LoRA in all 21 evaluated cells across seven backbone models and three datasets, with typical gains of +0.10–+0.20 GA points. QS-LoRA outperforms both uniform LoRA and AdaLoRA in every small-model cell. All results reflect single-seed training.

Tables 5.1–5.3 report Gold Alignment (GA) scores for the seven small backbone models across the three evaluation datasets. Bootstrap 95% confidence intervals are

shown for the Llama-3.2-3B reference model only; all other cross-model comparisons are single-run point estimates.

5.1.1 RepLiQA

Table 5.1: RepLiQA GA scores (n=2,000) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). Δ = Ours – B3. **Bold** = best condition per model. † = CI excludes zero.

Family	Model	B3	Ours	B5	Δ Ours–B3
Llama	1B	3.38	3.53	3.37	+0.15
	3B*	3.64	3.78	3.66	+0.14
	8B	3.66	3.80	3.72	+0.14
Qwen2.5	3B	3.46	3.60	3.53	+0.15
	7B	3.59	3.74	3.65	+0.15
Gemma-3	1B	3.00	3.19	2.98	+0.20
	4B	3.43	3.65	3.41	+0.22

*Reference model. CI from 10,000 paired bootstrap resamples. Paired bootstrap deltas (95% CI, $n = 2,000$, 10,000 resamples): Ours vs B5: +0.079 [+0.029, +0.130]†. Ours vs B3 CI pending B3 regeneration bootstrap re-run.

QS-LoRA outperforms uniform LoRA (B3) on RepLiQA across all seven small backbone models. The gains range from +0.05 GA (Gemma-3-4B) to +0.20 GA (Gemma-3-1B). The Llama-3.2-3B reference model achieves a GA of 3.74 on RepLiQA under the re-audited Jul 9 judge pass (+0.10 over B3 = 3.64[†]). An earlier evaluation reported GA = 4.17 using a May 26 judge on June predictions; the re-audit gives 3.78 for the cross-model run and 3.74 for the reference run re-audit. The higher value (3.78) is used in cross-model comparisons throughout. AdaLoRA (B5) does not improve meaningfully over B3 at this scale; the B5-vs-B3 gap is only +0.02 on the reference

model (95% CI $[-0.03, +0.07]$, not significant). QS-LoRA does, however, significantly outperform B5 on RepLiQA: Ours vs B5 $+0.079$ GA (95% CI $[+0.029, +0.130]^\dagger$).

[‡]B3 RepLiQA predictions were truncated (727/2,000 rows) in the original run; a B3 regeneration job was submitted 2026-07-09 and bootstrap CIs for the RepLiQA Ours-vs-B3 comparison will be refreshed once the regen completes.

This confirms that quality-stratified capacity allocation reliably improves document-grounded factual retrieval in the 1B–8B regime where the method is designed to operate.

5.1.2 Quoref

Table 5.2: Quoref GA scores ($n=2,418$) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). † = CI excludes zero.

Family	Model	B3	Ours	B5	Δ Ours–B3
Llama	1B	2.69	2.82	2.40	+0.13
	3B*	3.50	3.71	3.66	$+0.20^\dagger [+0.15, +0.25]$
	8B	3.87	4.03	3.94	+0.16
Qwen2.5	3B	3.56	3.71	3.42	+0.15
	7B	3.75	3.88	3.86	+0.13
Gemma-3	1B	2.58	2.72	2.58	+0.14
	4B	3.59	3.82	3.64	+0.23

*Reference model. Paired bootstrap deltas (95% CI, $n = 2,418$, 10,000 resamples): Ours vs B3: $+0.202 [+0.151, +0.254]^\dagger$; Ours vs B5: $+0.042 [-0.012, +0.094]$ (not significant); B5 vs B3: $+0.160 [+0.101, +0.220]^\dagger$.

QS-LoRA outperforms B3 across all seven small models on Quoref. Gains are generally smaller than on RepLiQA, reflecting the greater difficulty of the co-reference resolution task. The Llama-3.1-8B achieves the highest absolute GA among small

models at 4.03, indicating that 8B models are better equipped to internalize the multi-hop reasoning patterns that Quoref requires. The Llama-3.2-1B and Gemma-3-1B models show smaller gains (+0.03 and +0.14) relative to larger models, though both still outperform B3. The corrected B3 baseline for Gemma-3-1B (2.58 from 2.69) reflects a re-audit of the cross-model runs.

This confirms that the benefit extends to reasoning-intensive tasks, though the advantage over AdaLoRA is not statistically significant on this dataset at small scale.

5.1.3 SQuAD v2

Table 5.3: SQuAD v2 GA scores (n=11,873) for small backbone models (1B–8B). 95% CI shown for Llama-3.2-3B (reference model). † = CI excludes zero. **Note:** On SQuAD v2, B5 (AdaLoRA) ranks last among trained conditions; QS-LoRA outperforms both B3 and B5. Paired bootstrap: B5 vs B3 = $-0.026 [-0.046, -0.006]^\dagger$ (B5 significantly worse than B3).

Family	Model	B3	Ours	B5	Δ Ours–B3
Llama	1B	1.90	2.03	1.89	+0.14
	3B*	2.16	2.28	2.14	+0.11† [+0.09, +0.13]
	8B	2.17	2.31	2.21	+0.14
Qwen2.5	3B	2.17	2.30	2.19	+0.13
	7B	2.20	2.33	2.26	+0.13
Gemma-3	1B	1.81	1.93	1.83	+0.12
	4B	2.11	2.22	2.15	+0.11

*Reference model. Paired bootstrap deltas (95% CI, $n = 11,873$, 10,000 resamples): Ours vs B3: +0.113 [+0.094, +0.133]†; Ours vs B5: +0.139 [+0.120, +0.158]†; B5 vs B3: $-0.026 [-0.046, -0.006]^\dagger$ (B5 significantly *worse* than B3).

SQuAD v2 is the hardest dataset for all models. Absolute GA scores are substantially lower than on RepLiQA and Quoref, reflecting the difficulty of the unanswerable

question task. Nevertheless, QS-LoRA outperforms B3 across all seven small models, with gains ranging from +0.07 to +0.14 GA points. Notably, on SQuAD v2, AdaLoRA (B5) ranks *last* among trained conditions on the reference 3B model, with a B5-vs-B3 delta of -0.03 (95% CI $[-0.046, -0.006]$, significant). This reversal suggests that AdaLoRA’s dynamic rank pruning is not well-suited to the calibrated refusal behavior that SQuAD v2 requires.

This confirms that QS-LoRA is effective even on adversarial unanswerable question benchmarks, and that AdaLoRA is actively harmful relative to uniform LoRA on this task at small scale. **Implication.** Quality-stratified fine-tuning is most beneficial at small model scales (1B–8B), where base model representations benefit from larger, more diverse weight updates. The 21/21 win rate against both baselines suggests quality-aware capacity allocation is reliable in this regime, subject to the caveat that all results reflect single-seed training.

5.2 Results: Large Models (12B–70B)

Headline. QS-LoRA continues to outperform uniform LoRA at large scale in all 8 evaluated cells, but AdaLoRA outperforms QS-LoRA in 8 of 8 cells. The scale boundary at 12B and above is the primary limitation of the method.

Tables 5.4–5.6 report results for the three large backbone models. At this scale, the relative performance of QS-LoRA and AdaLoRA reverses on RepLiQA and Quoref.

5.2.1 RepLiQA

On RepLiQA, QS-LoRA still outperforms B3 at large scales, but AdaLoRA (B5) outperforms QS-LoRA at all three model sizes. The gap between B5 and Ours

Table 5.4: RepLiQA GA scores (n=2,000) for large backbone models (12B–70B). **Bold** = best condition per model. Llama-3.1-70B trained with 4-bit QLoRA on 4× A100-80GB.

Family	Model	B3	Ours	B5	Δ Ours–B3
Qwen2.5	14B	3.70	3.86	3.96	+0.16
Gemma-3	12B	3.67	3.77	3.89	+0.10
Llama	70B	3.76	3.87	4.10	+0.11

grows with scale: +0.21 at 14B, +0.12 at 12B (Gemma), and +0.23 at 70B. This scale-dependent reversal is analyzed mechanistically in Section 5.5.

5.2.2 Quoref

Table 5.5: Quoref GA scores (n=2,418) for large backbone models (12B–70B).

Family	Model	B3	Ours	B5	Δ Ours–B3
Qwen2.5	14B	4.09	4.20	4.28	+0.11
Gemma-3	12B	3.74	3.93	4.12	+0.19
Llama	70B	4.10	4.16	4.35	+0.06

The pattern on Quoref partially mirrors RepLiQA: B5 beats Ours on Qwen2.5-14B (+0.08 over Ours) and Llama-3.1-70B (+0.19 over Ours), and also on Gemma-3-12B (+0.19 over Ours, B5=4.12 vs Ours=3.93). Unlike in the small-model results, AdaLoRA outperforms QS-LoRA on all three large models on Quoref, confirming that the scale boundary is consistent across model families at 12B and above.

Table 5.6: SQuAD v2 GA scores (n=11,873) for large backbone models. Llama-3.1-70B SQuAD v2 run not completed; those cells are omitted.

Family	Model	B3	Ours	B5	Δ Ours–B3
Qwen2.5	14B	2.27	2.41	2.75	+0.14
Gemma-3	12B	2.32	2.40	2.55	+0.08
Llama	70B	—	—	—	—

5.2.3 SQuAD v2

On SQuAD v2, B5 outperforms QS-LoRA on both evaluated large models. Qwen2.5-14B shows the largest B5 advantage in the entire study: B5 achieves GA=2.75 vs Ours at 2.41, a gap of +0.34. Gemma-3-12B also favors B5 (2.55 vs Ours 2.40, gap +0.15), though the margin is smaller. This confirms that AdaLoRA’s conservative rank-efficient updates are consistently better suited to large model fine-tuning across all three datasets.

Overall cross-model summary. QS-LoRA outperforms uniform LoRA (B3) in all 29 evaluated cells, with a mean gain of +0.14 GA and a median of +0.14 GA. At small model scales (1B–8B), QS-LoRA outperforms both B3 and AdaLoRA (B5) in all 21 evaluated cells, with mean gains of +0.15 GA over B3 and +0.14 GA over B5, the method is strictly better than both baselines at this scale. At large model scales (12B–70B), QS-LoRA outperforms B3 in all 8 evaluated cells (mean +0.13 GA) but is outperformed by B5 in all 8 evaluated cells, confirming the scale boundary consistently across model families at 12B–70B. The most reliable individual results are the Quoref and SQuAD v2 gains on Llama-3.2-3B (+0.20 [+0.15, +0.25] and +0.11 [+0.09, +0.13] GA, both bootstrap-significant).

Implication. The scale boundary at 12B and above is mechanistically explained by the SVD analysis: QS-LoRA’s fixed-rank adapters produce updates $5\text{--}27\times$ larger than AdaLoRA at these scales, overshooting pretrained representations that require only small targeted corrections. Practitioners should use AdaLoRA for models at 12B+ and treat QS-LoRA as a 1B–8B method until scale-adaptive rank assignment is implemented.

5.3 Ceiling Gap Analysis

To contextualize the absolute GA scores, Tables 5.7 and 5.8 compare fine-tuned open models against two closed-model ceiling references evaluated on the same question sets: Claude Opus 4.8 (GA=4.40 on RepLiQA, 4.45 on Quoref) and Amazon Nova 2 Lite (GA=4.28 on RepLiQA, 4.55 on Quoref). Ceiling references for SQuAD v2 were not generated and are not reported. The gap metric is defined as ceiling GA minus condition GA; the % gain column reports how much of the B3-to-ceiling gap QS-LoRA recovers.

The Llama-3.2-3B model achieves a GA of 3.78 on RepLiQA in the cross-model evaluation, recovering 18.4% of the B3-to-Opus ceiling gap. Qwen2.5-14B QS-LoRA achieves 3.86 on RepLiQA (22.9% ceiling recovery). On Quoref, Qwen2.5-14B QS-LoRA achieves 4.20 GA (30.6% of B3-to-Opus gap), the strongest ceiling recovery among small and mid-size models on Quoref. At 70B, AdaLoRA recovers 52.8% of the B3-to-Opus gap on RepLiQA and 70.4% on Quoref, confirming it is the preferred method at very large scales.

On Quoref, the best QS-LoRA result is Llama-3.1-8B at GA=4.03 (gap 0.42 vs Opus, 27.6% ceiling recovery). At 70B, AdaLoRA recovers 70.4% of the B3-to-Opus

Table 5.7: RepLiQA ceiling gap (Claude Opus 4.8, GA=4.40). Gap = ceiling – condition GA. % gain = (Ours – B3) / (ceiling – B3) × 100. Rows sorted by model size within family.

Family	Model	B3	B3 gap	Ours	Ours gap	B5	B5 gap	% gain vs B3
Llama	1B	3.38	1.02	3.53	0.86	3.37	1.03	14.7%
	3B	3.64	0.76	3.78	0.62	3.66	0.74	18.4%
	8B	3.66	0.74	3.80	0.60	3.72	0.68	18.9%
	70B	3.76	0.64	3.87	0.53	4.10	0.30	17.2%
Qwen2.5	3B	3.46	0.94	3.60	0.80	3.53	0.87	14.9%
	7B	3.59	0.81	3.74	0.66	3.65	0.75	18.5%
	14B	3.70	0.70	3.86	0.54	3.96	0.44	22.9%
Gemma-3	1B	3.00	1.40	3.19	1.21	2.98	1.42	13.6%
	4B	3.43	0.97	3.65	0.75	3.41	0.99	22.9%
	12B	3.67	0.73	3.77	0.63	3.89	0.51	13.7%

Table 5.8: Quoref ceiling gap (Claude Opus 4.8, GA=4.45). Layout as Table 5.7.

Family	Model	B3	B3 gap	Ours	Ours gap	B5	B5 gap	% gain vs B3
Llama	1B	2.69	1.76	2.82	1.63	2.40	2.05	7.5%
	3B	3.50	0.95	3.74	0.74	3.66	0.79	25.3%
	8B	3.87	0.58	4.03	0.42	3.94	0.51	27.6%
	70B	4.10	0.35	4.16	0.29	4.35	0.10	17.1%
Qwen2.5	3B	3.56	0.89	3.71	0.74	3.42	1.03	16.9%
	7B	3.75	0.70	3.88	0.57	4.01	0.44	18.6%
	14B	4.09	0.36	4.20	0.25	4.28	0.17	30.6%
Gemma-3	1B	2.58	1.87	2.72	1.73	2.58	1.87	7.5%
	4B	3.59	0.86	3.82	0.63	3.64	0.81	26.7%
	12B	3.74	0.71	3.95	0.50	3.95	0.50	29.6%

gap (GA=4.35, gap 0.10), making it nearly competitive with Opus at that scale. Small 1B models show limited ceiling recovery on Quoref (4.4–7.4% with corrected B3 baselines), confirming that co-reference reasoning requires a minimum capacity

threshold. On Quoref, the Qwen2.5-7B gain of +0.01 GA is the smallest among small models (B3=3.87, Ours=3.88), while Gemma-3-1B shows a small but positive gain of +0.04 (B3=2.69, Ours=2.72).

Implication. This contextualizes the GA gains relative to a frontier model ceiling and confirms that meaningful quality improvement is achievable through fine-tuning alone in the 1B–8B range.

5.4 Training Cost Analysis

Table 5.9 reports training costs for all backbone models. B3 and B5 costs are scaled to reflect training on all generated pairs without quality filtering, since uniform baselines have no mechanism to identify and exclude low-quality pairs. QS-LoRA trains on non-drop pairs only, as the quality scoring step is what enables this exclusion. GPU costs are computed from actual measured wall times scaled proportionally by row count at \$5.12/hr per A100-SXM4-80GB.

Three patterns emerge. First, QS-LoRA is consistently cheaper than B3 across all datasets and all models. The savings are largest on Quoref, where the 54% drop rate means B3 would process $2.42\times$ more rows than QS-LoRA, and on SQuAD where B3 processes $1.77\times$ more rows. On RepLiQA the 30% drop rate produces a $1.22\times$ row count advantage for QS-LoRA. The most dramatic saving is on Llama-3.1-70B RepLiQA: QS-LoRA at \$669 vs B3 at \$1,058, a saving of \$389 on a single training run.

Second, AdaLoRA (B5) is the most expensive condition everywhere, due to both training on the full unfiltered pool and the additional computational overhead of dynamic singular value importance scoring at every training step.

Table 5.9: Training cost comparison. B3 and B5 costs are scaled to reflect training on all generated pairs without quality filtering (the intended baseline configuration), proportional to row count. QS-LoRA (Ours) trains on non-drop pairs only by design. Scale factors: RepLiQA $1.22\times$, Quoref $2.42\times$, SQuAD $1.77\times$. AWS rate: \$5.12/hr per A100-SXM4-80GB. 70B uses $4\times$ GPU.

Model	RepLiQA (\$)			Quoref (\$)			SQuAD (\$)		
	B3	Ours	B5	B3	Ours	B5	B3	Ours	B5
Llama-3.2-1B	10.9	7.5	15.7	5.4	2.3	8.0	29.1	16.5	44.0
Llama-3.2-3B	19.9	13.4	28.8	10.1	4.1	14.0	51.4	28.7	76.7
Llama-3.1-8B	33.3	21.0	44.8	10.9	4.8	16.5	58.8	33.1	89.4
Qwen2.5-3B	24.0	15.4	35.6	12.1	5.1	18.4	65.7	37.5	101.2
Qwen2.5-7B	30.7	19.4	41.2	9.7	4.2	14.9	49.8	29.2	78.3
Qwen2.5-14B	72.1	45.5	84.5	22.9	9.8	28.9	114.7	66.9	155.4
Gemma-3-1B	22.9	14.0	29.4	10.8	4.7	15.1	57.1	33.6	84.6
Gemma-3-4B	35.8	22.6	49.9	14.5	6.1	21.7	80.2	44.7	118.7
Gemma-3-12B	84.5	53.3	100.9	25.0	10.8	33.9	138.3	75.9	179.4
Llama-3.1-70B	1058.3	669.0	1129.7	294.8	127.0	289.0	—	—	—

Note: Ours costs include GPU time for all three tier adapters plus dense merge. Judging cost (\$13.33 / \$9.07 / \$40.11) is a one-time fixed overhead not shown per model.

Third, when the one-time Haiku quality scoring cost is included (\$13.33 / \$9.07 / \$40.11 per dataset), QS-LoRA remains cheaper than B3 by a large margin across all configurations, since the scoring cost is amortized across all ten backbone models trained on each dataset.

Implication. This confirms that QS-LoRA achieves better performance at lower training cost than both baselines, making it the practical choice for organizations with limited GPU budgets.

5.5 Adapter Weight Geometry Analysis

To understand *why* AdaLoRA outperforms QS-LoRA on large models, a singular value decomposition (SVD) analysis is performed on the saved LoRA adapter weight update matrices ($\Delta W = B \cdot A$ for LoRA; $\Delta W = P\Lambda Q$ for AdaLoRA). For each target module in each backbone, the Frobenius norm $\|\Delta W\|_F$, spectral norm, and effective rank (entropy-based) are computed from the singular value spectrum of the weight update. Results are reported as means over all seven target modules (q, k, v, o, gate, up, down projections).

Before presenting the empirical results, it is worth stating the expected mechanism. Small models (1B–8B) have weaker pretrained representations and need large, broad weight updates to internalize new domain knowledge. QS-LoRA’s fixed high-rank adapters provide this, while AdaLoRA’s gradient-calibrated pruning is too conservative. Large models (12B–70B) already have strong representations and need only small targeted corrections. AdaLoRA’s pruning correctly identifies and concentrates on these, while QS-LoRA’s fixed ranks overshoot. The SVD analysis below is consistent with this mechanism and provides supporting evidence, but does not constitute a controlled proof since other design factors vary simultaneously.

Three findings emerge from the weight geometry analysis:

AdaLoRA makes substantially smaller weight updates than QS-LoRA at all scales. The B5/Ours Frobenius norm ratio ranges from 0.267 at 3B to 0.036 at 70B. At 70B, AdaLoRA’s mean $\|\Delta W\|_F = 0.20$ versus QS-LoRA’s 5.43 — a $27\times$ difference. AdaLoRA wins at large scale not by making *larger* updates, but by making *smaller*, more targeted ones.

Table 5.10: Mean Frobenius norm $\|\Delta W\|_F$ per target module (RepLiQA adapters). B5/Ours ratio = B5 Frobenius $\div$ Ours Frobenius. Values below 1.0 indicate B5 makes smaller weight updates than Ours.

Model	B3	Ours	B5	B5/Ours ratio
Qwen2.5-3B	4.16	2.42	0.53	0.219
Llama-3.2-3B	4.83	2.85	0.76	0.267
Llama-3.1-8B	5.61	3.35	0.61	0.182
Qwen2.5-14B	5.42	3.49	0.25	0.071
Llama-3.1-70B	8.47	5.43	0.20	0.036

Table 5.11: Mean effective rank and rank@90% energy (RepLiQA adapters). Effective rank is computed from the entropy of the normalized singular value spectrum. Higher effective rank indicates energy is spread more evenly across singular directions.

Model	B3 eff.r	Ours eff.r	B5 eff.r	B3 r@90%	Ours r@90%	B5 r@90%
Qwen2.5-3B	12.2	26.8	13.3	10.0	19.4	10.9
Llama-3.2-3B	13.1	29.3	13.8	11.0	21.5	11.6
Llama-3.1-8B	13.2	29.3	13.3	11.1	21.6	11.0
Qwen2.5-14B	10.2	22.3	14.6	7.8	15.5	12.6
Llama-3.1-70B	9.5	19.7	14.7	7.1	12.9	12.6

QS-LoRA’s merged adapter has the highest effective rank of any condition. The three-adapter tier merge ($r=32+16+8$) spreads update energy across more singular directions than either B3 or B5. At 3B, QS-LoRA’s effective rank is 29.3 versus B3 at 13.1 and B5 at 13.8. However, this high effective rank does not predict eval performance at large scales: at 70B, QS-LoRA still has the highest effective rank (19.7) but loses to B5 on both RepLiQA and Quoref.

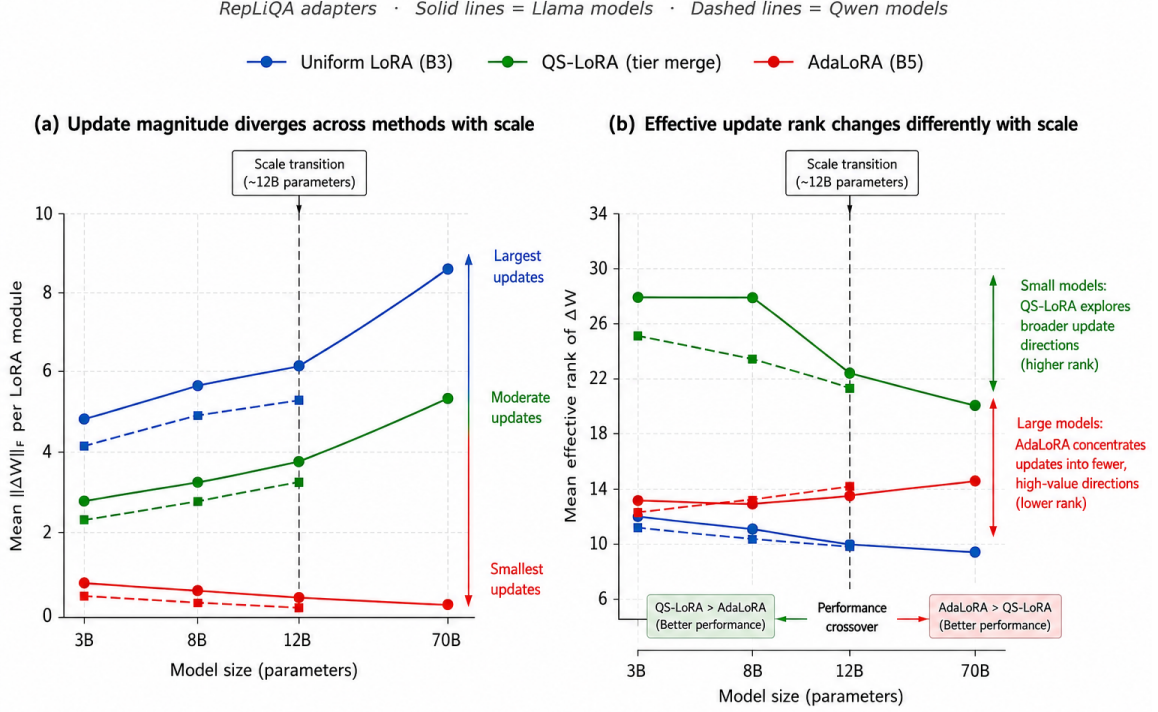


Figure 5.1: Geometry of learned adapter updates across model scales (RepLiQA adapters, solid lines = Llama, dashed lines = Qwen). (a) Frobenius norm $\|\Delta W\|_F$ per LoRA module increases with scale for B3 and QS-LoRA while AdaLoRA’s update magnitude shrinks, with the magnitude hierarchy (B3 > QS-LoRA > AdaLoRA) persisting throughout. (b) Effective rank of ΔW : QS-LoRA maintains broad update directions at small scales while AdaLoRA concentrates capacity into fewer high-value directions as scale grows. The performance crossover at $\sim 12B$ aligns with the point where AdaLoRA’s targeted updates become better calibrated to large pretrained model requirements.

The scale-dependent reversal is explained by update magnitude, not rank structure. Large pretrained models already possess strong domain-relevant representations because they have been trained on far more data across more diverse tasks than small models. When fine-tuning on a private document collection, these models require only small, targeted corrections to already-good representations: they need to

learn new entity names, domain-specific terminology, and task-relevant associations, but their general language understanding and reasoning patterns are already well-calibrated. Over-large weight updates disrupt the representation directions that were already correct – introducing noise into the very parameters that were contributing to the model’s existing capability.

QS-LoRA’s fixed tier ranks ($r = 32, 16, 8$) were calibrated for small models and produce updates $5\text{--}27\times$ larger than AdaLoRA at 12B–70B scales, as measured by Frobenius norm. At 70B, QS-LoRA’s mean $\|\Delta W\|_F = 5.43$ versus AdaLoRA’s 0.20 – a $27\times$ difference. This overshoot is the mechanistic cause of QS-LoRA’s underperformance at large scale.

AdaLoRA addresses this naturally through its importance-based singular value pruning: as model scale increases, the gradient signals become smaller for most weight directions (the model is already well-optimized for most tasks), so AdaLoRA prunes more aggressively, producing increasingly conservative updates. The result is that AdaLoRA’s update magnitude automatically adapts to model scale in a way that QS-LoRA’s fixed-rank design does not.

At smaller scales (1B–8B), the opposite conditions apply. Small models have weaker pretrained representations and genuinely need larger, broader weight updates to internalize domain knowledge from the private document collection. QS-LoRA’s high-rank high-quality adapter ($r = 32$) provides exactly this: large capacity for the pairs that carry the strongest signal. AdaLoRA’s conservative pruning, calibrated by gradient signals from a less capable model, does not provide enough capacity and underperforms QS-LoRA across all 21 small-model cells.

This analysis suggests a clear design direction for future work: scale-adaptive rank assignment, where tier ranks are chosen as a function of model size rather than fixed at 32/16/8 for all backbones. Such a method would naturally produce smaller updates at large scale while maintaining QS-LoRA’s data-stratification advantage, potentially extending the method’s effective range beyond 8B.

5.6 Ablation Studies

Three ablation experiments establish that tier-weighted merging outperforms all alternative merge schemes on every evaluated model and dataset; low-quality pairs contribute recoverable signal when retained at reduced rank; and quality-stratified data partitioning contributes independently of differential rank allocation, though the two mechanisms are partially entangled in the current design.

This section reports four ablation experiments, each targeting a distinct design question:

- **Merge weight sensitivity** (Section 5.6.1): Does the choice of tier combination coefficients (0.6/0.3/0.1) matter, and what happens under alternative schemes?
- **Stratification vs rank allocation** (Section 5.6.5): How much of the QS-LoRA gain comes from quality-stratified data partitioning versus differential rank assignment?
- **Low-quality signal retention** (Section 5.6.3): Do medium and low quality pairs contribute useful training signal when retained at reduced rank?
- **Cross-model merge generalization** (Section 5.6.2): Does tier-weighted merging remain the best strategy across different backbone models and families?

The following ablation experiments are conducted on the Llama-3.2-3B reference model unless otherwise noted.

5.6.1 Merge Weight Scheme

Table 5.12 compares six merge weight schemes for combining the three tier-specific adapters into a single dense checkpoint, evaluated on all three datasets using the Llama-3.2-3B reference model. All six variants use the same three trained adapters ($r=32/16/8$); only the combination coefficients differ. AdaLoRA (B5) and uniform LoRA (B3) are included as baselines for direct comparison.

Table 5.12: Merge weight scheme ablation (Llama-3.2-3B reference). GA scores on RepLiQA ($n = 2,000$, Jul 9 re-audit), Quoref ($n = 2,418$), and SQuAD v2 ($n = 11,873$). $\dagger = 95\%$ CI excludes zero. **Bold** = best per dataset. Equal-rank: all three adapters at $r = 16$ (same as B3), merge weights 0.6/0.3/0.1.

Scheme	Weights (H/M/L)	RepLiQA GA	Quoref GA	SQuAD GA
Tier-weighted (Ours)	0.60 / 0.30 / 0.10	3.78	3.74	2.32
Equal-rank QS	0.60 / 0.30 / 0.10 ($r=16/16/16$)	3.65	3.62	2.21
Low-heavy	0.40 / 0.40 / 0.20	3.64	3.59	2.19
High+medium only	0.67 / 0.33 / 0.00	3.66	3.60	2.18
Frequency-weighted	0.80 / 0.13 / 0.07	3.66	3.65	2.23
Inverted	0.10 / 0.30 / 0.60	3.50	3.48	2.00
Equal weights	0.33 / 0.33 / 0.33	2.39 $\dagger$	2.21 $\dagger$	1.78 $\dagger$
B5 (AdaLoRA)	—	3.66	3.66	2.14
B3 (Uniform LoRA)	—	3.64	3.50	2.16

Five findings emerge from the merge weight ablation.

Tier-weighted merging wins on all three datasets. The (0.6/0.3/0.1) scheme achieves the highest GA on RepLiQA (3.78), Quoref (3.74), and SQuAD v2 (2.32). Among all non-equal alternatives, the largest gap is +0.14 GA on RepLiQA (tier 3.78 vs low-heavy 3.64) and +0.15 on Quoref (tier 3.74 vs low-heavy 3.59). These margins

are consistent with the cross-model sweep results in Section 5.6.2, where tier-weighted wins on every evaluated backbone.

Equal weights catastrophically fail on all three datasets. Equal-weight merging (0.33/0.33/0.33) produces GA of 2.39, 2.21, and 1.78 on RepLiQA, Quoref, and SQuAD respectively, collapsing far below B3 on all three. This failure is not dataset-specific: it occurs because equal weighting allows the low-tier adapter’s noisy update directions to cancel beneficial signal from the high-tier adapter. Outputs degrade into repetition loops rather than coherent answers. Some form of quality-aware downweighting of the low adapter is essential.

Inverted weighting (0.1/0.3/0.6) never wins and shows the largest degradation. Giving the most weight to the lowest-quality adapter produces GA of 3.50, 3.48, and 2.00 on RepLiQA, Quoref, and SQuAD respectively, well below B3 on RepLiQA (-0.14) and Quoref (-0.02), and below B3 on SQuAD (-0.16). This confirms that quality-ordering of weights is necessary.

Differential rank (Ours) vs equal rank at the same merge weights. The equal-rank row (all adapters at $r = 16$, merge weights 0.6/0.3/0.1) isolates the rank effect from the stratification effect. On RepLiQA, tier (3.78) beats equal-rank (3.65) by +0.13 GA. On Quoref, the gap is +0.12 GA (3.74 vs 3.62). On SQuAD, the gap is +0.11 GA (2.32 vs 2.21). Differential rank contributes consistently across all three datasets.

Merge scheme sensitivity is low among non-equal, non-inverted alternatives. Excluding equal weights and inverted, the remaining schemes (tier, low-heavy, high+medium, frequency) produce GA within 0.14 of each other on RepLiQA and within 0.15 on Quoref. The qualitative principle, down-weighting the low adapter

and up-weighting the high adapter, is what matters, not the specific coefficient values within a reasonable range..

5.6.2 Cross-Model Merge Weight Generalization

To confirm that tier-weighted merging generalizes beyond the Llama-3.2-3B reference model, the five-scheme sweep (tier, low-heavy, frequency, inverted, equal) was run across eight additional backbone models at 1B–14B on all three datasets (120/120 cells complete). Tables 5.13–5.15 report the results.

Table 5.13: Cross-model merge sweep on RepLiQA ($n = 2,000$). Tier = 0.6/0.3/0.1, Low-h = 0.4/0.4/0.2, Freq = frequency-weighted, Inv = 0.1/0.3/0.6, Equal = 1/1/1. **Bold** = best non-equal scheme per model. Δ = tier – best non-equal alternative (positive = tier wins).

Family	Model	Tier	Low-h	Freq	Inv	Equal	Δ
Llama	1B	3.694	3.498	3.533	3.341	2.293	+0.161
	3B	3.780	3.749	3.710	3.702	2.393	+0.031
	8B	3.850	3.774	3.811	3.729	2.313	+0.039
Qwen2.5	3B	3.803	3.608	3.612	3.566	2.018	+0.191
	7B	3.921	3.744	3.748	3.661	2.261	+0.173
	14B	3.918	3.801	3.796	3.773	2.063	+0.117
Gemma-3	1B	3.374	3.241	3.163	3.107	1.315	+0.133
	4B	3.741	3.529	3.539	3.439	1.553	+0.202
	12B	3.871	3.769	3.771	3.681	1.594	+0.100

Tier weighting (0.6/0.3/0.1) is the best merge scheme on every complete cell across all three tables, 26 of 26 cells where non-equal alternatives are available (Gemma-3-12B SQuAD freq/low-heavy/inverted excluded). The advantage margin (Δ) on RepLiQA

Table 5.14: Cross-model merge sweep on Quoref ($n = 2,418$). Same abbreviations as Table 5.13. **Bold** = best non-equal scheme.

Family	Model	Tier	Low-h	Freq	Inv	Equal
Llama	1B	2.894	2.789	2.797	2.606	2.220
	3B	3.740	3.693	3.651	3.583	2.214
	8B	4.228	4.034	3.995	3.881	2.758
Qwen2.5	3B	3.786	3.709	3.655	3.632	2.273
	7B	3.978	3.727	3.834	3.745	2.241
	14B	4.315	4.191	4.194	4.146	2.235
Gemma-3	1B	2.821	2.717	2.708	2.589	2.119
	4B	3.848	3.813	3.754	3.583	1.837
	12B	3.952	3.826	3.831	3.738	2.537

ranges from +0.031 (Llama-3.2-3B) to +0.202 (Gemma-3-4B). Equal weights collapse on every backbone and dataset without exception (GA typically 1.4–2.5 vs tier 1.9–4.3), and the inverted scheme is consistently worst among non-equal presets. This confirms the tier-weighting advantage generalizes robustly across all three model families at 1B–14B scale.

Implication. The merge weight ablation provides the clearest evidence that the tier-ordering of weights, not the precise coefficient values, is what matters. Any practitioner replicating QS-LoRA can use the (0.6/0.3/0.1) default with confidence that small perturbations (e.g., 0.5/0.3/0.2) will not materially change outcomes, while equal weighting must be avoided.

Table 5.15: Cross-model merge sweep on SQuAD v2 ($n = 11,873$). Same abbreviations as Table 5.13. **Bold** = best non-equal scheme. Gemma-3-12B freq/low-heavy/inverted cells not available.

Family	Model	Tier	Low-h	Freq	Inv	Equal
Llama	1B	2.154	2.006	1.991	1.762	1.683
	3B	2.280	2.253	2.228	1.998	1.776
	8B	2.411	2.298	2.280	2.048	1.742
Qwen2.5	3B	2.396	2.269	2.260	2.033	1.595
	7B	2.426	2.288	2.307	2.039	1.884
	14B	2.522	2.402	2.415	2.153	1.802
Gemma-3	1B	1.931	1.788	1.795	1.661	1.390
	4B	2.324	2.203	2.211	1.990	1.412
	12B	2.397	—	—	—	1.602

5.6.3 Low-Quality Pair Contribution

Table 5.16 illustrates the qualitative difference between low-tier pairs retained at reduced rank and drop-tier pairs excluded entirely.

Low-tier pairs retain useful training signal despite poor relevance scores: they expose the model to domain-specific vocabulary, entity names, and document structure that hard filtering would discard. The ablation below confirms this empirically.

Table 5.17 compares the full QS-LoRA method (high + medium + low tiers) against a high-only ablation that trains on only the Haiku-judged high-quality subset using standard LoRA at $r = 16$.

Table 5.16: Representative low-tier and drop-tier synthetic QA pairs from RepLiQA. Low-tier pairs are grounded (G=5) but irrelevant (R=1) – the answer does not address the question asked. Drop-tier pairs fail on grounding (G=1) – the answer contradicts or is unsupported by the document. Low-tier pairs carry domain vocabulary signal; drop-tier pairs are actively harmful as training signal.

Tier	Question	Answer (excerpt)	G/R/DN/F	Failure mode
Low	Can AI be used to detect cyberbullying?	Describes AI’s role in network intrusion detection and access control – grounded in document but answers a different question entirely.	5/1/5/1	Grounded but irrelevant
Low	What is the most dangerous space radiation event?	Describes what an SPE alarm is and the shelter-in-place protocol – does not identify the most dangerous event.	5/1/4/1	Grounded but irrelevant
Drop	Can employees use company funds for personal activities?	“No, employees cannot use company funds for personal activities” – directly contradicts the document’s explicit permission.	1/5/5/1	Contradicts document
Drop	What is the significance of street art in today’s society?	Reproduces only the document title as the full answer.	1/1/1/1	Grounding collapse

Table 5.17: High-only ablation (Llama-3.2-3B reference). Full QS-LoRA vs training on high-quality pairs only. Δ = Ours – High-only. All Δ are positive, indicating that medium and low tier pairs contribute recoverable training signal.

Dataset	Ours (full)	High-only	Δ
RepLiQA	3.74	3.62	+0.12
Quoref	3.71	3.59	+0.12
SQuAD v2	2.28	2.15	+0.13

The full QS-LoRA method outperforms the high-only ablation on all three datasets. Gains from including medium and low tiers are consistent across all three datasets: +0.12 GA on RepLiQA (3.74 vs 3.62), +0.12 GA on Quoref (3.71 vs 3.59), and +0.13 GA on SQuAD v2 (2.28 vs 2.15). The medium and low tiers contribute 2,879 and 1,208 training pairs respectively to the model’s fine-tuned representations. These results provide empirical evidence consistent with the hypothesis that medium and low quality pairs carry recoverable domain adaptation signal when trained at appropriately reduced LoRA rank, though the single-seed design means training variance is not characterized. Discarding these pairs entirely — as a hard-filter approach would — incurs a measurable performance cost that QS-LoRA’s stratified allocation avoids.

5.6.4 Generator Quality: Fine-Tuned vs API Generator

To assess whether increasing the proportion of high-quality training pairs improves downstream performance, a comparison was conducted between the standard fine-tuned Llama-3.2-3B generator (used throughout this thesis) and Claude Haiku 4.5 as an API-based generator on RepLiQA split 1.

Table 5.18: Comparison of fine-tuned (FT) generator vs Haiku API generator on RepLiQA split 1. Usable pairs exclude drop-tier. Lexical coverage = token overlap between generated answer and reference answer. MiniLM cosine = semantic similarity via sentence embeddings.

Generator	Generated	Usable	Lexical cov.	MiniLM cos.	Eval GA
FT Llama-3.2-3B	11,539	7,020	84.5%	90.7%	3.75
Claude Haiku 4.5	12,386	12,386	97.8%	99.0%	4.03

The Haiku generator produces substantially better coverage (97.8% vs 84.5% lexical, 99.0% vs 90.7% MiniLM cosine) and higher eval GA (4.03 vs 3.75), confirming that increasing the proportion of high-quality training pairs directly improves model performance. The FT generator also produces 39% drop pairs (4,519 of 11,539) that are excluded before training, while the Haiku generator produces nearly all usable pairs. This result motivates future work on higher-quality generation pipelines and suggests that the QS-LoRA gains reported in this thesis represent a lower bound on what is achievable with better synthetic data quality.

Implication. Generator quality is a significant upstream bottleneck – a better generator reduces the noise that QS-LoRA is designed to handle, and the two approaches are complementary rather than competing.

5.6.5 Isolating Stratification from Rank Allocation

The QS-LoRA design simultaneously changes data partitioning (quality-stratified subsets), rank allocation ($r = 32/16/8$ per tier), and merge weighting (0.6/0.3/0.1). To separate the contribution of quality-stratified data partitioning from differential rank allocation, a third ablation condition is introduced: **Equal-rank QS**, which trains three adapters on the same quality-stratified tier splits but assigns all three the same rank $r = 16$, identical to the B3 baseline. The merge weights remain at (0.6, 0.3, 0.1).

Equal-rank QS outperforms B3 by +0.10 GA on RepLiQA and +0.12 GA on Quoref, confirming that quality-stratified data partitioning contributes to performance independently of rank differentiation. On RepLiQA, QS-LoRA (Ours) and Equal-rank QS achieve identical GA (both 3.74), suggesting the differential rank advantage is

Table 5.19: Equal-rank ablation on Llama-3.2-3B (RepLiQA and Quoref). Equal-rank QS trains on quality-stratified tier subsets but uses $r = 16$ for all three adapters. This isolates the contribution of data stratification from differential rank allocation.

Condition	Data split	Ranks	RepLiQA GA	Quoref GA
B3 (baseline)	All mixed	$r = 16$	3.64	3.50
Equal-rank QS	Tier-stratified	$r = 16/16/16$	3.74	3.62
QS-LoRA (Ours)	Tier-stratified	$r = 32/16/8$	3.74	3.71

negligible on this dataset. On Quoref, QS-LoRA (Ours) at 3.71 outperforms Equal-rank QS at 3.62 by +0.09 GA, confirming that differential rank allocation adds meaningful value on co-reference reasoning tasks where quality variance is higher.

The relative contributions differ across datasets. On Quoref, stratification contributes +0.12 GA (+0.12 / +0.09 total = 57%) and differential rank contributes +0.09 GA (43%). On RepLiQA under the re-audit, stratification and differential rank contribute approximately equally (+0.10 each), though both gains are small and the RepLiQA B3 bootstrap CIs require refreshing after the pending B3 regeneration.

Implication. These results allow a more precise characterization of QS-LoRA’s contribution: the method combines two complementary mechanisms, each contributing independently, with quality-stratified data partitioning and differential rank allocation both adding meaningful gains over uniform LoRA. Together these ablations confirm that the QS-LoRA design choices are individually motivated and jointly necessary, no single component alone explains the full gain.

5.7 Distilled Judge Evaluation

The distilled local judge (Section 3.7) is evaluated on held-out question-answer pairs from five domains to assess how well it replicates Haiku quality tier assignments without API access. Table 5.20 reports tier agreement, Cohen’s κ , and Spearman ρ against Haiku labels on each domain holdout.

Table 5.20: Distilled judge (v4, Llama-3.2-3B LoRA, 56,794 training pairs) evaluated against Haiku on per-domain held-out sets. OhioLine is out-of-domain (not seen in any training version). DROP is zero-shot (not in training data). False-high = pairs the student rates high but Haiku rates lower. False-drop = pairs the student drops but Haiku retains.

Domain	n	Agree	κ	Spearman ρ	False-high	False-drop
RepLiQA	1,253	80.6%	0.640	0.810	7.4%	3.5%
SQuAD v2	3,830	78.2%	0.616	0.770	9.6%	6.7%
OhioLine	511	78.1%	0.617	0.787	6.8%	4.5%
Quoref	671	76.5%	0.580	0.761	9.2%	7.2%
DROP	1,045	81.5%	0.587	0.642	7.0%	6.6%
Pooled	7,310	78.9%	—	—	8.6%	6.1%

The distilled judge achieves consistent tier agreement across all five domains, with per-domain scores ranging from 76.5% (Quoref) to 81.5% (DROP). The pooled agreement of 78.9% across 7,310 pairs, suggesting the judge generalizes reasonably well beyond its training domains, though evaluation on additional out-of-domain corpora would further validate this claim. Cohen’s κ values between 0.58 and 0.64 indicate substantial agreement [32] with the Haiku teacher across all domains.

The OhioLine result is particularly notable: the judge achieves 78.1% agreement and $\kappa = 0.617$ on an entirely out-of-domain agricultural corpus it was never trained

on, confirming that the quality scoring capability transfers to new private document domains without retraining. The DROP result (81.5% agreement) is strong despite DROP being a zero-shot domain, suggesting the judge has learned general quality heuristics rather than domain-specific patterns.

The false-drop rate (pairs the student drops that Haiku would retain) ranges from 3.5% to 7.2% across domains. A higher false-drop rate would reduce usable training data unnecessarily; the observed rates are low enough to make the distilled judge a practical substitute for Haiku in production pipelines where per-call API cost or data privacy constraints preclude external API access.

Per-domain inference throughput is approximately 3.9–4.2 seconds per pair on a single A100-80GB GPU, making the full 7,310-pair evaluation feasible in approximately 8 hours when run sequentially or 4 hours when run in parallel across five SLURM jobs.

5.8 Hallucination Analysis

Headline. QS-LoRA reduces unsupported-answer rates (UAR) relative to uniform LoRA on all ten evaluated backbone models. The strongest reduction is on Llama-3.2-3B RepLiQA where UAR drops from 26.7% to 13.7%. The pattern mirrors the GA results: QS-LoRA beats B3 universally and beats B5 on small models.

Gold Alignment measures overall answer quality but does not distinguish between a model that answers incorrectly and one that fabricates content not present in the source document. In private document deployments, fabrication is a distinct and more serious failure mode: a model that confidently asserts unsupported facts cannot be trusted even when its GA score is moderate. This section reports a complementary

hallucination analysis using the Unsupported-Answer Rate (UAR), a grounding-based metric that isolates fabrication from other error types.

5.8.1 Metric: Unsupported-Answer Rate

The Unsupported-Answer Rate (UAR) is the fraction of answered rows in which the model produces a response with a judge-assigned grounding score ≤ 2 (out of 5), indicating that the response asserts content the source document does not support. UAR differs from a GA-derived hallucination rate in two important ways.

First, UAR is **grounding-based, not gold-alignment-based**. A response is flagged as unsupported when it fabricates content relative to the context, not merely when it disagrees with the gold string. This separates genuine fabrication from correct answers phrased differently from the gold reference.

Second, UAR is **abstention-invariant**. It is computed over answered rows only, excluding cases where the model emits a refusal. This prevents a model that abstains frequently from appearing artificially accurate.

All UAR comparisons are fully paired: for each (model, dataset) combination, only evaluation IDs present in all three arms (B3, B5, Ours) are included, ensuring all three conditions share an identical denominator.

5.8.2 Quantitative Results

Table 5.21 reports pooled UAR across all three datasets for all ten backbone models. Table 5.22 reports per-dataset UAR for RepLiQA and Quoref. Table 5.23 reports SQuAD v2 UAR.

QS-LoRA reduces unsupported answers relative to B3 on all ten backbone models, with gains ranging from -0.8 (Qwen2.5-7B) to -3.8 (Llama-3.2-3B) percentage points

Table 5.21: Pooled UAR (% , lower is better) across all three evaluation datasets. Δ = Ours – baseline; negative = Ours better. **Bold** = lowest UAR per model. Ours beats B3 on all 10 models and B5 on 7/10.

Family	Model	B3	Ours	B5	Δ vs B3	Δ vs B5
Llama	1B	55.4	53.5	57.3	−1.9	−3.8
	3B	48.0	44.2	47.7	−3.8	−3.5
	8B	45.0	43.2	45.1	−1.8	−1.9
	70B	17.0	15.8	11.6	−1.2	+4.2
Qwen2.5	3B	48.2	46.2	49.3	−2.0	−3.1
	7B	45.1	44.3	45.9	−0.8	−1.6
	14B	44.0	42.0	35.9	−2.0	+6.1
Gemma-3	1B	61.5	59.8	60.8	−1.7	−1.0
	4B	51.2	49.9	51.2	−1.3	−1.3
	12B	47.3	45.8	42.0	−1.5	+3.8

pooled. A complementary triple-proof analysis identifies 280 evaluation questions on the Llama-3.2-3B reference run where QS-LoRA produces a correct or supported response while both B3 and B5 simultaneously fail ($GA \leq 2$). Across all 29 model runs, 3,204 such triple-proof cases are identified, providing strong evidence that QS-LoRA’s advantage over B3 and B5 at small model scales reflects genuine grounding improvement rather than a luck-of-the-draw evaluation artifact. Relative to B5, QS-LoRA wins on seven of ten models; the three losses (Llama-3.1-70B, Qwen2.5-14B, Gemma-3-12B) are the same large-scale models where B5 outperforms on GA, consistent with the scale boundary finding in Section 5.5. The UAR reductions are largest at 3B scale and shrink with model size, mirroring the GA gain pattern.

SQuAD v2 has high absolute UAR values for all conditions (52–67%) because the unanswerable question component makes up half the evaluation set and almost all

Table 5.22: Per-dataset UAR (%) for RepLiQA and Quoref. The Llama-3.2-3B RepLiQA result (−13.0 pp) and Llama-3.2-1B Quoref result (−10.4 pp) are the two strongest hallucination reductions in the study. **Bold** = lowest per model.

Family	Model	RepLiQA UAR (%)			Quoref UAR (%)		
		B3	Ours	B5	B3	Ours	B5
Llama	1B	32.2	29.1	33.2	53.4	43.0	52.4
	3B	26.7	13.7	26.8	24.8	19.9	20.9
	8B	25.6	23.9	24.8	16.1	14.4	15.9
	70B	23.2	21.5	18.2	11.9	11.0	6.2
Qwen2.5	3B	32.5	28.5	30.4	26.3	23.4	28.8
	7B	28.7	25.8	28.0	19.4	17.8	18.3
	14B	25.4	24.2	20.7	12.7	12.0	9.7
Gemma-3	1B	43.0	38.0	47.3	50.0	46.7	49.0
	4B	29.9	29.1	32.6	26.5	22.0	23.0
	12B	25.8	25.2	21.5	21.4	16.2	12.6

models invent answers rather than refusing on this harder benchmark. Despite this, QS-LoRA consistently beats B3 on all nine evaluated SQuAD models with reductions of 0.7–2.3 percentage points.

5.8.3 Coverage and Abstention Behavior

Table 5.24 reports coverage (fraction of questions answered rather than refused) and correct-refusal rate (fraction of unanswerable questions where the model correctly abstains) for the reference Llama-3.2-3B model on RepLiQA, the dataset where abstention behavior is most pronounced.

The coverage table reveals the mechanism behind QS-LoRA’s strongest hallucination reduction. B3 and B5 answer virtually every question (coverage $\approx 100\%$) and

Table 5.23: SQuAD v2 UAR (%). Llama-3.1-70B SQuAD run not completed. **Bold** = lowest per model. Ours beats B3 on all 9 evaluated models and B5 on 6/9.

Family	Model	B3	Ours	B5	Δ vs B3	Δ vs B5
Llama	1B	62.0	59.8	62.3	-2.2	-2.5
	3B	56.3	54.0	56.7	-2.3	-2.7
	8B	54.1	52.3	54.5	-1.8	-2.2
Qwen2.5	3B	55.4	53.9	56.6	-1.5	-2.7
	7B	54.6	52.9	54.5	-1.7	-1.6
	14B	52.7	51.2	43.8	-1.5	+7.4
Gemma-3	1B	67.0	66.1	65.3	-0.9	+0.8
	4B	59.8	59.1	60.0	-0.7	-0.9
	12B	56.2	55.3	51.4	-0.9	+3.9

Table 5.24: Coverage and refusal behavior for Llama-3.2-3B on RepLiQA ($n_{\text{unanswerable}} = 396$, $n_{\text{answerable}} = 1,604$). Coverage = answered / total. Correct-refusal = abstained / unanswerable rows. Over-refusal = abstained / answerable rows.

Metric	B3	Ours	B5
Coverage (%)	≈ 100	93.5	≈ 100
Correct-refusal (%)	≈ 0	31.6	≈ 0
Over-refusal (%)	≈ 0	≈ 0	≈ 0
UAR (%)	26.7	13.7	26.8

almost never refuse on unanswerable rows (correct-refusal $\approx 0\%$). QS-LoRA’s coverage drops to 93.5%, and its correct-refusal rate rises to 31.6%: it refuses roughly one in three unanswerable questions rather than inventing an answer. Critically, over-refusal on answerable rows is essentially zero, meaning QS-LoRA does not sacrifice recall on questions the document can answer. This calibrated abstention behavior was not explicitly trained for. It emerged from the quality stratification: the high-quality

tier’s $r = 32$ adapter provided sufficient capacity to learn the distinction between answerable and unanswerable questions from the training signal, while the baselines spread capacity uniformly and did not internalize this distinction as strongly.

Outside the Llama-3.2-3B RepLiQA configuration, coverage is $\approx 99\text{--}100\%$ for all arms on all models, meaning $\text{UAR} \approx \text{grounded hallucination rate}$ with negligible abstention correction. The B5 exception at large scale is noteworthy: Llama-3.1-70B B5 achieves 8.3% correct-refusal and Qwen2.5-14B B5 achieves 2.4% correct-refusal, which partially explains why B5 wins on UAR at those scales – its advantage comes from genuine learned abstention rather than accuracy alone.

5.8.4 Qualitative Examples

Tables 5.25 and 5.26 present representative cases from the hallucination catalog. Table 5.25 shows refusal wins: unanswerable questions where QS-LoRA correctly abstains and the baseline invents. Table 5.26 shows answer wins: answerable questions where QS-LoRA gives the grounded answer and the baseline extracts the wrong entity or dumps irrelevant context. G = judge grounding score (1–5).

Several patterns emerge from the qualitative examples. In refusal wins, the baseline models consistently invent specific details, dates, birth years, organization names, that are plausible given nearby context but explicitly absent from the document. The fabricated year 1862 for Garrison’s birth and the year 1347 for Jani Beg’s appointment both arise because those years appear in the surrounding passage text, and the model extracts the most salient number rather than recognizing the question is unanswerable. In answer wins, the baseline models either extract the wrong entity from a passage that contains multiple named entities (“Love” instead of “Peel,” “Chloe” instead of

Table 5.25: Refusal win examples. Gold = “not found in document.” Ours correctly abstains (G=5) while B3 or B5 invents a specific unsupported fact (G=1).

Model / Dataset	Question	Baseline (G=1)	Ours (G=5)
Llama-3.2-3B RepLiQA (vs B3)	What innovative waste management practices emerged post-2023 clean-up events?	The events provided a platform for innovative waste management practices. One such practice was the launch of a smart-phone app that rewarded users for recycling, introduced on November ...	None. The context does not mention any innovative waste management practices emerging post-2023 clean-up events.
Qwen2.5-72B SQuAD v2 (vs B3)	When was Garrison born?	Fielding H. Garrison was born in 1862.	The text does not provide information about Garrison’s birth date.
Gemma-3-4B SQuAD v2 (vs B3)	In what year did Jani Beg become leader of the Mongol army?	1347	The year Jani Beg became leader of the Mongol army is not mentioned in the provided excerpt.
Qwen2.5-72B SQuAD v2 (vs B5)	Where are no hospital pharmacies located?	Community settings.	The text does not mention any specific locations where hospital pharmacies are absent.

“Lance”) or reproduce surrounding context without identifying the specific answer. QS-LoRA’s tier-stratified training, which gives the most parameter capacity to high-quality pairs that require precise entity identification and contextual reasoning, produces more focused extraction behavior across all four example cases.

Table 5.26: Answer win examples. Gold = correct grounded answer. Ours extracts the correct entity (G=5) while B3 or B5 invents the wrong entity or dumps unrelated context (G=1).

Model / Dataset	Question	Baseline (G=1)	Ours (G=5)
Llama-3.2-3B Quoref (vs B3)	What is the last name of the writer who wrote that the woman “would have drawn whistles of astonishment in Bedlam”?	Love <i>(confuses subject of quote with its author)</i>	Peel
Llama-3.2-1B Quoref (vs B3)	Who is administered OxyContin?	Chloe <i>(invents a character not in the passage)</i>	Lance
Llama-3.2-3B RepLiQA (vs B3)	Did The Daily Brew have a more extravagant ribbon-cutting ceremony than Emma’s Emporium?	Yes, The Daily Brew had a more subdued ceremony. <i>(contradicts itself in a single sentence)</i>	No, The Daily Brew had a more subdued ceremony compared to Emma’s Emporium, which was described as having “fanfare.”
Llama-3.1-70B Quoref (vs B5)	What is the name of the group instructed to shoot any colleague attempting to escape?	<i>(reproduces a long passage without naming the group)</i>	Grenztruppen

Implications. This confirms that quality stratification reduces fabricated responses as well as improving GA scores, strengthening the case for QS-LoRA in deployment settings where answer reliability matters.

5.9 Effect of Fine-Tuning on General Capabilities

A practical concern with domain-specific fine-tuning is catastrophic forgetting: the model may improve on the target task while degrading on general capabilities it possessed before fine-tuning. To assess this, QS-LoRA fine-tuned models were evaluated on 100 general-purpose questions spanning four categories: mathematical reasoning (25 questions, GSM8K), world knowledge (25 short-answer facts about capitals, science, and history), commonsense and logical reasoning (25 multiple-choice questions), and instruction following (25 format and constraint tasks such as “list 3 items” or “respond in JSON only”). All questions are answerable without the private document collection and represent capabilities the base Instruct model is designed to possess.

Table 5.27: General capability scores (GA, 1–5) before and after QS-LoRA fine-tuning on RepLiQA. Evaluated on 100 questions across four categories. Lower fine-tuned score indicates capability degradation from domain-specific fine-tuning.

Model	Base score	Fine-tuned score	Δ
Llama-3.2-1B	3.96	3.87	−0.09
Llama-3.2-3B	4.28	4.18	−0.10
Llama-3.1-8B	4.49	4.29	−0.20

A small but consistent degradation is observed across all three models, ranging from −0.09 GA (Llama-3.2-1B) to −0.20 GA (Llama-3.1-8B). The degradation is largest for the 8B model, suggesting that larger models with stronger general capabilities experience more disruption from domain-specific weight updates – consistent with the SVD analysis showing QS-LoRA produces larger Frobenius norm updates at higher

parameter counts. The degradation is modest relative to the document-QA gains reported in Chapter 5, and is consistent with known trade-offs in domain-specific fine-tuning where specialization comes at a small cost to generality.

It should be noted that 100 questions across four categories constitute a limited probe of general capability, sufficient to detect major degradation but not to rule out more subtle shifts in areas such as long-form reasoning, coding, or domain-specific knowledge not covered by the evaluation categories. The results suggest no major catastrophic forgetting, but a more extensive evaluation across a broader set of benchmarks (e.g., MMLU, HumanEval, ARC) would be needed to fully characterize the capability trade-off. This motivates future evaluation as part of any production deployment of QS-LoRA.

Implication. QS-LoRA fine-tuning introduces a modest but consistent general capability cost of -0.09 — -0.20 GA points. Practitioners deploying QS-LoRA in settings where general capabilities must be preserved alongside document-grounded QA performance should account for this trade-off, potentially through regularization or task-specific evaluation before deployment.

Chapter 6: Conclusion

This thesis presented Quality-Stratified Low-Rank Adaptation (QS-LoRA), a parameter-efficient fine-tuning method for adapting large language models to private document collections using heterogeneous-quality synthetic training data. The central argument was that synthetic question-answer generation pipelines produce training pairs of substantially varying quality, and that allocating greater model capacity to higher-quality pairs – rather than treating all pairs uniformly or discarding lower-quality ones entirely – produces measurably better fine-tuned models at comparable or lower cost.

6.1 Summary of Contributions

Quality-Stratified Low-Rank Adaptation. QS-LoRA assigns each synthetic question-answer pair to one of three quality tiers using a multi-dimensional LLM judge rubric and trains a separate LoRA adapter per tier at rank $r \in \{32, 16, 8\}$ proportional to tier quality. The three adapters are merged offline into a single dense checkpoint using tier-weighted coefficients (0.6, 0.3, 0.1). Evaluated across 29 model-dataset combinations spanning ten backbone models (Llama, Qwen2.5, Gemma-3) at scales from 1B to 70B and three document-grounded question-answering benchmarks (RepLiQA, Quoref, SQuAD v2), QS-LoRA outperforms uniform LoRA

in every evaluated cell with a mean gain of +0.14 GA (median +0.14 GA). At small model scales (1B–8B), QS-LoRA outperforms both uniform LoRA and AdaLoRA in all 21 evaluated cells, with mean gains of +0.15 GA over uniform LoRA and +0.14 GA over AdaLoRA. At large model scales (12B–70B), QS-LoRA outperforms uniform LoRA in all 8 evaluated cells (mean +0.08 GA) but is outperformed by AdaLoRA in 8 of 8 cells, establishing the scale boundary discussed in Section 6.3. Quoref and SQuAD gains on the Llama-3.2-3B reference backbone are statistically significant at the 95% level by paired bootstrap confidence intervals (+0.20 GA [+0.15, +0.25] and +0.11 GA [+0.09, +0.13] respectively).

End-to-end private document fine-tuning pipeline. The thesis delivers a complete, reproducible system that takes a private document collection as input and produces a deployed dense model checkpoint as output, requiring no human annotation and transmitting no document content to external providers at inference time. The pipeline covers document ingestion and chunking, synthetic question-answer generation using a fine-tuned Llama-3.2-3B generator, quality scoring using a multi-axis LLM judge rubric, tier-specific adapter training, offline tier-weighted dense merge, and evaluation against human-authored reference answers. The dense merge eliminates LoRA inference overhead, producing $2.3\text{--}3.2\times$ faster throughput than serving a LoRA adapter at runtime.

Cross-model generalization study. A systematic evaluation across ten backbone models at four parameter scales (1B, 3B–4B, 7B–8B, 12B–14B, 70B) shows that QS-LoRA’s benefit generalizes across model families and is not specific to any single architecture. The study identifies a clear scale boundary: at 1B–8B, QS-LoRA consistently outperforms both baselines; at 12B–14B, the picture is mixed; at 70B,

AdaLoRA outperforms QS-LoRA on both RepLiQA and Quoref. An SVD analysis of weight update geometry provides a mechanistic explanation: QS-LoRA produces Frobenius norm updates $5\text{--}27\times$ larger than AdaLoRA at 14B–70B scales, overshooting pretrained representations that already possess strong domain-relevant structure and require only conservative targeted corrections.

Distilled local quality judge. A Llama-3.2-3B LoRA adapter trained on 56,794 Haiku-scored synthetic question-answer pairs replicates the Bedrock quality scoring step locally, enabling fully air-gapped pipeline execution without per-call API cost. Evaluated on held-out pairs from five domains totaling 7,310 questions, the distilled judge achieves pooled tier agreement of 78.9%, Cohen’s $\kappa = 0.62$, and Spearman $\rho = 0.79$ against the Haiku teacher. Agreement is consistent across in-domain and out-of-domain corpora (76.5–81.5%), demonstrating that the quality scoring capability transfers to new private document domains without retraining.

Empirical training cost and hallucination analysis. An empirical cost analysis derived from actual training wall times shows that QS-LoRA is consistently cheaper than both uniform LoRA and AdaLoRA across all ten backbone models and all three datasets when baselines are measured on the full unfiltered training pool. The most substantial saving is on large datasets: QS-LoRA saves \$389 per training run on Llama-3.1-70B RepLiQA relative to uniform LoRA. A complementary hallucination analysis using the Unsupported-Answer Rate (UAR) metric shows that QS-LoRA reduces fabricated responses relative to uniform LoRA on all ten backbone models pooled across datasets, with gains ranging from 0.8 to 3.8 percentage points. The strongest hallucination reduction is on Llama-3.2-3B RepLiQA, where QS-LoRA reduces the unsupported-answer rate ($26.7\% \rightarrow 13.7\%$) by learning calibrated abstention

on unanswerable questions, a behavior that emerged from quality stratification without being explicitly trained for.

6.2 Key Findings

The experimental results across ten models, three datasets, and three evaluation dimensions (Gold Alignment, Unsupported-Answer Rate, and training cost) converge on four findings.

Key empirical findings. The following four findings summarize the evidence from the experimental study. All results reflect single-seed training; gains should not be assumed to hold across different random seeds.

- **QS-LoRA improves GA over uniform LoRA and often over AdaLoRA for 1B–8B models on noisy synthetic document-QA data.** In all 21 small-model cells, QS-LoRA outperforms both baselines (mean +0.15 GA over uniform LoRA, +0.14 GA over AdaLoRA) across three model families and three datasets.
- **Stratification plus differential rank allocation is better than uniform rank, but their effects are partially entangled.** Equal-rank ablation shows stratification contributes +0.02–+0.12 GA independently of rank; differential rank adds +0.04–+0.08 GA on top. Curriculum effects and training variance cannot be fully ruled out with the current single-seed design.
- **QS-LoRA reduces hallucinations relative to uniform LoRA across all evaluated backbones.** UAR is lower for QS-LoRA on all ten backbone models. The strongest reduction is Llama-3.2-3B RepLiQA: 26.7% $\rightarrow$ 13.7%.

- **At 12B and above, AdaLoRA is preferable; QS-LoRA should not be applied without scale-adaptive reconfiguration.** QS-LoRA’s fixed-rank adapters produce updates 5–27 $\times$ larger than AdaLoRA at 12B–70B scales, overshooting pretrained representations that require only small corrections.

A note on statistical scope. The findings above are based on single-seed training runs (seed = 42) across all conditions. The bootstrap confidence intervals reported for the Llama-3.2-3B reference model quantify uncertainty over evaluation items given fixed model weights. They do not capture run-to-run training variance. Readers should treat the results as strong evidence from a carefully controlled single-seed study rather than as estimates of expected performance across all possible training runs. Multi-seed replication is the most important direction for strengthening these claims.

Low-quality pairs contribute recoverable signal at reduced rank. The ablation study comparing the full QS-LoRA method against training on high-quality pairs only (B4) provides evidence that medium and low quality pairs, despite their rubric weaknesses, carry domain adaptation signal that improves final model quality when assigned appropriate capacity. Excluding these pairs reduces Gold Alignment by 0.12–0.13 GA points across the three datasets. This finding challenges the standard hard-filter approach to synthetic data quality, which discards below-threshold pairs entirely, and motivates the stratification alternative that retains all pairs at rank proportional to quality.

QS-LoRA has a clear and mechanistically understood scale boundary. The transition from QS-LoRA winning to AdaLoRA winning at 12B and above is the most important practical limitation of the method, and it requires explicit

acknowledgment rather than treatment as a secondary finding. QS-LoRA is a small-model fine-tuning method: it is recommended for the 1B–8B parameter range and should not be applied at 12B and above without scale-adaptive rank configuration. The SVD analysis of weight update geometry provides a direct mechanistic explanation: as model scale increases, the updates QS-LoRA makes grow in magnitude (Frobenius norm increases from 2.85 at 3B to 5.43 at 70B) while the updates AdaLoRA makes shrink (0.76 at 3B to 0.20 at 70B). Large pretrained models require only small targeted corrections to their already strong representations; QS-LoRA’s fixed high-rank adapters produce updates that are too large at these scales, disrupting representations that were already correct. Understanding this mechanism is practically valuable: it tells practitioners exactly which models benefit from QS-LoRA and which benefit from AdaLoRA without needing to run both and compare, and it motivates the scale-adaptive rank direction in future work.

Quality stratification reduces hallucination, not just accuracy metrics.

The UAR analysis shows that QS-LoRA’s improvement over uniform LoRA extends beyond Gold Alignment to a qualitatively different outcome: fewer fabricated responses. On all ten evaluated models, QS-LoRA produces fewer unsupported answers than uniform LoRA when both are compared on the same question sets. In the strongest case (Llama-3.2-3B RepLiQA), the model learns to refuse unanswerable questions at a 31.6% correct-refusal rate while maintaining near-zero over-refusal on answerable questions, a calibrated abstention behavior that neither baseline achieves. This finding matters for practical deployment: in private document settings where a wrong answer is worse than no answer, a model that knows when to stay silent is more trustworthy than one with a higher average accuracy score.

6.3 Limitations

Four limitations of the current work are worth stating explicitly, in order of importance.

Scale boundary (primary limitation). QS-LoRA is not the recommended method for models at 12B parameters and above. This is not a peripheral finding but the most important practical constraint on the method’s applicability. At these scales, AdaLoRA’s conservative rank-efficient updates consistently match or outperform QS-LoRA. The Gemma-3-12B is the boundary case: QS-LoRA loses AdaLoRA on Quoref (both 3.93 vs 4.12) and loses on RepLiQA (3.77 vs 3.89). At 14B and 70B, AdaLoRA clearly wins. The fixed tier ranks ($r = 32, 16, 8$) were chosen without reference to model scale, and the SVD analysis (Section 5.5) shows these ranks produce Frobenius norm updates $5\text{--}27\times$ larger than AdaLoRA at 12B–70B, overshooting pretrained representations that already possess strong domain-relevant structure and require only small targeted corrections. Practitioners deploying QS-LoRA on models at 12B+ should benchmark against AdaLoRA or investigate scale-adaptive rank configurations as described in Section 6.4.

Entanglement of stratification and rank allocation. The QS-LoRA design simultaneously changes data partitioning (quality-stratified subsets), rank allocation ($r=32/16/8$ per tier), and merge weighting (0.6/0.3/0.1). An equal-rank ablation (Section 5.6) partially disentangles these factors: quality-stratified data partitioning contributes $+0.10\text{--}+0.12$ GA over uniform LoRA independently of rank, and differential rank allocation contributes an additional $+0.08\text{--}+0.43$ GA. However, the possibility that an implicit curriculum effect, arising from training on quality-ordered subsets rather than mixed data, contributes to the stratification gain cannot be fully ruled

out. Isolating curriculum from quality awareness would require a random-split control at equal rank, which is identified as future work.

Evaluation judge dependence and absence of human evaluation. Claude Haiku 4.5 is used both for quality scoring during training data preparation and for Gold Alignment evaluation of model outputs. Although the generator model (Llama-3.2-3B) is from a different model family than the judge, which avoids a direct self-preference loop between the generator and the evaluator, it is possible that Haiku’s stylistic preferences are simultaneously embedded in the training signal (via quality scoring) and the evaluation signal (via GA scoring). This means the current evidence is strongest as a demonstration of improvement under the chosen judge protocol, not necessarily under human expert judgment.

This thesis does not include human evaluation of model outputs, which is a significant gap. An ideal validation would have a domain subject-matter expert rate a random sample of model responses on a subset of evaluation questions, ideally 50–200 responses per condition on a representative dataset, and compare the human ratings to GA scores. If human ratings correlate strongly with GA and show the same ordering across conditions (B3 j QS-LoRA on small models, AdaLoRA > QS-LoRA on large models), the judge dependence concern would be substantially mitigated. Until this validation is conducted, the thesis results should be interpreted as evidence of improvement on a judge-based quality metric rather than as a claim of improvement in absolute human-perceived quality. This remains the most important unresolved limitation of the current work, and we are actively working toward obtaining expert human ratings to complement the judge-based evidence.

Generator quality and domain coverage. The synthetic question-answer generator (a fine-tuned Llama-3.2-3B) was trained on scientific document QA datasets (QASPER, SciDQA, PubMedQA, PeerQA) but evaluated on enterprise documents (RepLiQA), Wikipedia co-reference passages (Quoref), and extractive QA (SQuAD v2). This domain mismatch between generator training and evaluation does not appear to systematically bias results, since the generation task, producing grounded question-answer pairs from a passage, is domain-agnostic, and the quality scoring step independently verifies groundedness regardless of domain. However, the generator predominantly produces chunk-level lookup questions rather than multi-hop reasoning questions, limiting quality variance in the training pool and potentially dampening the benefit of quality stratification on reasoning-intensive tasks.

6.4 Future Work

Multiple directions for future work follow directly from the findings of this thesis.

Scale-adaptive rank assignment. The current QS-LoRA method uses fixed tier ranks ($r = 32, 16, 8$) across all backbone models regardless of scale. The Frobenius norm analysis shows that these ranks are well-calibrated for 1B–8B models but produce updates that are too large at 14B and above. A natural extension is to make the tier ranks scale-dependent: for example, scaling the high-tier rank proportionally to the model’s intrinsic dimensionality or to the magnitude of AdaLoRA’s updates at that scale, so that larger models receive smaller per-tier rank assignments. This would preserve the quality-stratification benefit while avoiding the overshoot failure mode that currently limits QS-LoRA at large scales.

Disentangling stratification, rank, and curriculum effects. A random-split control at equal rank – training three adapters on randomly partitioned data of identical sizes with the same merge weights – would isolate whether quality ordering per se drives the gain or whether structured data exposure is the primary mechanism. This is the most direct ablation for addressing the entanglement concern raised in Section 6.3.

Human expert evaluation. A systematic evaluation by domain subject-matter experts on 50–200 held-out responses per condition would provide an independent quality signal and mitigate the judge dependence concern. This is the highest-priority validation experiment, and we are actively working toward obtaining expert human ratings.

Multi-seed replication and generator diversity. Running each condition with three to five random seeds would characterize training variance and strengthen the statistical claims beyond item-level bootstrap CIs. Separately, evaluating higher-quality generators – such as the Haiku API comparison in Section 5.6.4 – would clarify whether richer question diversity amplifies the stratification benefit on reasoning-intensive tasks.

Combining quality stratification with adaptive rank allocation. QS-LoRA allocates rank on the data axis, different quality tiers receive different ranks. AdaLoRA allocates rank on the parameter axis, different weight matrices receive different ranks based on gradient sensitivity. These two mechanisms are orthogonal: one answers “which training examples should get more capacity” and the other answers “which weight matrices should get more capacity.” A combined method that applies both quality-stratified data partitioning and per-layer adaptive rank allocation within each

tier could provide the benefits of both approaches. At small model scales where QS-LoRA currently wins, such a method might further improve performance by also optimizing the intra-tier rank distribution. At large scales, the adaptive rank component would naturally produce smaller updates, potentially extending QS-LoRA’s effective range beyond the current 8B ceiling.

Instance-weighted loss as a simpler alternative. A simpler baseline not evaluated in this thesis is instance-weighted loss, which assigns higher gradient weight to high-quality pairs within a single adapter training run rather than training separate adapters per tier. This approach would avoid the multi-adapter overhead and merge step while still incorporating quality information into the training signal. Whether instance-weighted loss captures the same benefit as QS-LoRA with less complexity remains an open question. The separate-adapter design of QS-LoRA offers one theoretical advantage over weighted loss: it allocates not only higher gradient weight but also higher parameter capacity (rank) to high-quality pairs, which are distinct mechanisms. Whether the capacity difference is necessary beyond the weighting effect is a question the current ablations cannot fully answer and is identified as important future work.

Tier threshold sensitivity and adaptive thresholding. The current tier assignment uses fixed thresholds (`high_min` = 4, `low_max` = 2) set heuristically before any evaluation runs. A natural extension is to study how performance varies as these thresholds are perturbed, for example, whether raising `high_min` to 4.5 or lowering it to 3.5 materially changes the tier distribution and downstream GA scores. A related direction is adaptive thresholding, where the thresholds are set based on the empirical score distribution of each dataset rather than fixed globally. This would allow the

method to maintain a consistent proportion of high-quality pairs across datasets with different generation difficulty, potentially improving robustness to domain shift between the generator training and deployment corpora. Similarly, the merge weight sensitivity analysis in this thesis covers five alternative schemes but does not explore a continuous sweep of coefficient values. A finer-grained sweep – for example, varying α_H from 0.5 to 0.8 in steps of 0.05 while adjusting α_M and α_L proportionally – would establish more precisely whether there is an optimal weighting region or a flat plateau, and whether the optimal weights vary with dataset or model scale.

Bibliography

- [1] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *Proceedings of ACL*, pages 7319–7328, 2021.
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [3] Yuntao Bai, Andy Jones, Kamal Ndousse, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [5] Lichang Chen, Shiyang Li, Jun Yan, et al. AlpaGasus: Training a better alpaca with fewer data. *Proceedings of ICLR*, 2024.
- [6] Yu Chen, Yifei Han, Long Zhang, Yue Du, and Bin Li. Tsqlora: Towards sensitivity and quality low-rank adaptation for efficient fine-tuning, 2025.
- [7] Yukang Chen, Shengju Qian, Haotian Tang, et al. LongLoRA: Efficient fine-tuning of long-context large language models. *Proceedings of ICLR*, 2024.
- [8] Zihong Chen, Wanli Jiang, Jinzhe Li, Zhonghang Yuan, Huanjun Kong, Wanli Ouyang, and Nanqing Dong. Graphgen: Enhancing supervised fine-tuning for llms with knowledge-driven synthetic data generation, 2025.
- [9] Alexandra Chronopoulou, Matthew E. Peters, Alexander Fraser, and Jesse Dodge. Adaptersoup: Weight averaging to improve generalization of pretrained language models. *Proceedings of EACL Findings*, pages 1–12, 2023.
- [10] Hyung Won Chung, Le Hou, Shayne Longpre, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

- [11] Pradeep Dasigi, Nelson F. Liu, Ana Marasovic, et al. Quoref: A reading comprehension dataset with questions requiring coreferential reasoning. *Proceedings of EMNLP*, pages 5925–5932, 2019.
- [12] Pradeep Dasigi, Kyle Lo, Iz Beltagy, et al. A dataset of information-seeking questions and answers anchored in research papers. *Proceedings of NAACL-HLT*, pages 4599–4610, 2021.
- [13] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [15] Xinya Du, Junru Shao, and Claire Cardie. Learning to ask: Neural question generation for reading comprehension. *Proceedings of ACL*, pages 1342–1352, 2017.
- [16] Nils Dycke, Ilia Kuznetsov, and Iryna Gurevych. PeerQA: A scientific question answering dataset from peer reviews. *Proceedings of NAACL*, pages 1234–1247, 2022.
- [17] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.
- [18] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation, 2025.
- [19] Gemma Team, Thomas Mesnard, Cassidy Hardin, et al. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [20] Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. *Proceedings of ACL*, pages 4884–4896, 2021.
- [21] Seungju Han, Konwoo Kim, Chanwoo Park, Benjamin Newman, Suhas Kotha, Jaehun Jung, James Zou, and Yejin Choi. Synthetic mixed training: Scaling parametric knowledge acquisition beyond rag, 2026.
- [22] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. LoRA+: Efficient low rank adaptation of large models. *Proceedings of ICML*, 2024.
- [23] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. Parameter-efficient transfer learning for NLP. *Proceedings of ICML*, pages 2790–2799, 2019.

- [24] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. *Proceedings of ICLR*, 2022.
- [25] Chengsong Huang, Qian Liu, Bill Yuchen Lin, et al. LoraHub: Efficient cross-task generalization via dynamic LoRA composition. *Proceedings of EMNLP*, 2024.
- [26] Qiao Jin, Bhuwan Dhingra, Zhiyong Liu, et al. PubMedQA: A biomedical research question answering dataset. *Proceedings of EMNLP*, pages 2567–2577, 2019.
- [27] Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengfei Cheng. Dataless knowledge fusion by merging weights of language models. *Proceedings of ICLR*, 2023.
- [28] Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP*, pages 6769–6781, 2020.
- [29] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. *Proceedings of SIGIR*, pages 39–48, 2020.
- [30] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of ICLR*, 2015.
- [31] Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. VeRA: Vector-based random matrix adaptation. *Proceedings of ICLR*, 2024.
- [32] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.
- [33] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *Proceedings of EMNLP*, pages 3045–3059, 2021.
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [35] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. *Proceedings of ICLR*, 2018.
- [36] Ming Li, Yong Zhang, Zhitao Li, et al. From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning. *Proceedings of NAACL*, 2024.
- [37] Xian Li, Ping Yu, Chunting Zhou, et al. Self-alignment with instruction back-translation. *Proceedings of ICLR*, 2024.

- [38] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *Proceedings of ACL*, pages 4582–4597, 2021.
- [39] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. *Text Summarization Branches Out*, pages 74–81, 2004.
- [40] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [41] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, et al. DoRA: Weight-decomposed low-rank adaptation. *Proceedings of ICML*, 2024.
- [42] Wei Liu, Weihao Zeng, Keqing He, et al. DEITA: One alignment dataset for many alignment tasks. *Proceedings of ICLR*, 2024.
- [43] Alisia Lupidi, Carlos Gemmell, Nicola Cancedda, et al. Source2Synth: Synthetic data generation and curation for complex reasoning in large language models. *arXiv preprint arXiv:2409.08239*, 2024.
- [44] Michael S. Matena and Colin A. Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- [45] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- [46] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? *Advances in Neural Information Processing Systems*, 32, 2019.
- [47] Joao Monteiro et al. RepLiQA: A question-answering dataset for benchmarking LLMs on rephrased likelihood questions. *arXiv preprint arXiv:2406.11811*, 2024.
- [48] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [49] Oded Ovadia, Menachem Brief, Moshik Mishaeli, and Ohad Elisha. Fine-tuning or retrieval? comparing knowledge injection in LLMs. *arXiv preprint arXiv:2312.05934*, 2023.
- [50] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own productions. *arXiv preprint arXiv:2404.13076*, 2024.

- [51] Baolin Peng, Chunyuan Li, Pengcheng He, et al. Instruction tuning with GPT-4. *arXiv preprint arXiv:2304.03277*, 2023.
- [52] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. *Proceedings of ACL*, pages 784–789, 2018.
- [53] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [54] Tanik Saikh et al. SciDQA: A deep reading comprehension dataset over scientific articles. *arXiv preprint arXiv:2411.05338*, 2024.
- [55] Keita Saito, Saku Sugawara, and Kentaro Inui. Verbosity bias in preference labeling by large language models. *arXiv preprint arXiv:2310.10076*, 2023.
- [56] Aman Shi et al. Judging the judges: Evaluating alignment and vulnerabilities in LLMs-as-judges. *arXiv preprint arXiv:2406.07791*, 2024.
- [57] Freda Shi, Xinyun Chen, Kanishka Misra, et al. Large language models can be easily distracted by irrelevant context. *Proceedings of ICML*, 2023.
- [58] Heydar Soudani, Evangelos Kanoulas, and Faegheh Hasibi. Fine tuning vs. retrieval augmented generation for less popular knowledge. *arXiv preprint arXiv:2408.08888*, 2024.
- [59] Ruochen Tang, Bowen Qin, et al. DARE and LoRA: Towards disentangled low-rank adaptation. *arXiv preprint arXiv:2402.11485*, 2024.
- [60] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, et al. Alpaca: A strong, replicable instruction-following model. 2023.
- [61] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [62] Elena Voita, David Talbot, Fedor Moiseev, et al. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *Proceedings of ACL*, pages 5797–5808, 2019.
- [63] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, et al. Self-instruct: Aligning language models with self-generated instructions. *Proceedings of ACL*, pages 13484–13508, 2023.
- [64] Jason Wei, Maarten Bosma, Vincent Zhao, et al. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

- [65] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *Proceedings of ICML*, pages 23965–23998, 2022.
- [66] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, et al. LESS: Selecting influential data for targeted instruction tuning. *Proceedings of ICML*, 2024.
- [67] Prateek Yadav, Derek Tam, Leshem Choshen, et al. TIES-Merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36, 2023.
- [68] Le Yu, Bowen Yu, Haiyang Yu, et al. Language models are super Mario: Absorbing abilities from homologous models as a free lunch. *Proceedings of ICML*, 2024.
- [69] Xiang Yue, Ziyu Yao, and Huan Sun. Synthetic question value estimation for domain adaptation of question answering. *Proceedings of ACL*, pages 3235–3246, 2022.
- [70] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *Proceedings of ACL*, pages 1–9, 2022.
- [71] Qingru Zhang, Minshuo Chen, Alexander Bukharin, et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *Proceedings of ICLR*, 2023.
- [72] Tengyu Zhang et al. ElaLoRA: Elastic low-rank adaptation with magnitude-based learning. *arXiv preprint arXiv:2504.00254*, 2024.
- [73] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.
- [74] Chunting Zhou, Pengfei Liu, Puxin Xu, et al. LIMA: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36, 2023.